



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文开题报告

报告题目 基于智能体的漏洞挖掘与修复技术研究

学生姓名 张小北 学号 202518018629028

指导教师 侯锐 职称 研究员

学位类别 工学博士

学科专业 计算机系统结构

研究方向 漏洞挖掘与修复

培养单位 中国科学院信息工程研究所

填表日期 2026 年 09 月

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 选题背景	1
1.2 选题意义	3
二 国内外本学科领域的发展现状与趋势	4
2.1 基于智能体的语义漏洞挖掘	4
2.2 基于智能体的漏洞自动修复	14
2.3 基于智能体的漏洞修复验证	22
三 课题主要研究内容与预期目标	28
3.1 研究思路	28
3.2 研究内容一：基于智能体的语义漏洞挖掘方法	31
3.3 研究内容二：基于智能体的漏洞自动修复方法	32
3.4 研究内容三：基于智能体的漏洞修复验证方法	33
3.5 预期目标与创新点	34
四 拟采用的研究方法、技术路线、实验方案及其可行性分析 ..	34
4.1 基于智能体的语义漏洞挖掘方法	34
4.2 基于智能体的漏洞自动修复方法	36
4.3 基于智能体的漏洞修复验证方法	38
4.4 实验方案设计	40
五 已有科研基础与所需的科研条件	42
5.1 已有科研基础	42
5.2 所需科研条件	42
六 研究工作计划与进度安排	42

一 选题的背景及意义

1.1 选题背景

1.1.1 漏洞攻防格局正在发生深刻变化

随着软件规模持续增长与软件供应链日益复杂，安全漏洞的数量、影响面与利用速度都在快速上升。一方面，国家级漏洞库中公开漏洞数量逐年攀升，攻击者利用公开漏洞信息实施“N-day 快速武器化”的窗口期不断缩短；另一方面，大量部署中的二进制组件缺乏透明、可验证的补丁状态，使下游用户与监管方难以判断“这个漏洞到底有没有被修复”。以 Log4j（CVE-2021-44228）为代表的事件表明：单一组件缺陷可以引发整个生态的修复验证难题 [1]。

与此同时，以大语言模型（LLM）为核心的**智能体（Agent）**技术正以前所未有的速度进入软件安全领域。与传统“静态规则 + 人工操作”的分析模式不同，智能体通过“模型推理 + 工具调用 + 迭代反馈”的闭环，能够自主完成代码阅读、二进制分析、测试用例生成、补丁合成等过去高度依赖专家经验的任务。Google Project Zero 与 DeepMind 的 Big Sleep 项目首次展示了 AI 智能体在真实开源项目中发现未知内存安全漏洞的能力 [2]；DARPA AIxCC 大赛则系统验证了智能体在“自动发现—自动修补—自动验证”全链路中的可行性 [3]。**智能体正从“辅助工具”演变为漏洞攻防闭环中的核心行动者。**

1.1.2 漏洞攻防闭环中的三个关键环节

一个完整的漏洞攻防闭环，可以抽象为三个相互衔接的核心环节：

1. **漏洞挖掘（Discovery）**：在目标软件中发现未知漏洞，回答“哪里可能有洞”。
2. **漏洞修复（Automated Vulnerability Repair, AVR）**：针对已确认存在的漏洞，自动定位并合成补丁，回答“怎么把洞补上”。
3. **漏洞修复验证（Repair Verification）**：判定修复是否真正消除漏洞根因、目标程序是否实际包含修复，回答“洞补上了没有、补得对不对”。

这三个环节覆盖了漏洞从“未知 → 已知 → 修复”的完整生命周期，彼此共享大量基础能力：对漏洞语义的理解、对补丁意图的建模、对二进制与源码之间

语义鸿沟的桥接、对修复正确性的判定。**然而，现有研究大多将三者割裂对待**，分别由不同团队、不同工具链、不同基准独立推进，导致漏洞语义、补丁语义、修复语义在各自环节中被重复建模，既浪费资源，也阻碍了跨环节能力的复用。

1.1.3 智能体带来的新机遇与新问题

智能体技术的引入，为统一上述三个环节提供了可能：

- **能力侧**：前沿编码智能体已能熟练调用 binutils、Ghidra、gdb 等二进制与源码分析工具，完成多步推理与决策。
- **成本侧**：token 成本与推理延迟的持续下降，使”在真实仓库级代码库上大规模运行智能体”从理想变为可行。
- **生态侧**：SWE-bench [4]、CVE-Bench [5]、AIxCC [3] 等评测基准的成熟，为系统化比较不同智能体安全能力提供了基础设施。

但与此同时，三个关键环节在智能体视角下仍各自存在未被充分回答的核心问题：

- 在漏洞挖掘上，传统 fuzzing 与符号执行在语义/逻辑漏洞（如认证绕过、状态机错误、配置驱动缺陷）上进展缓慢，**核心难点是”自因变量”——那些不受外部输入控制、由内部状态/配置/时序决定的变量——导致现有输入驱动方法在错误维度上爆炸。**
- 在漏洞修复上，智能体已能生成”看起来合理”的补丁，但”**补丁是否真正消除了漏洞根因**”这一判定问题（即 PPT 在修复场景下的对偶问题）仍缺乏可靠机制，导致现有基准上”补丁过拟合 PoC、回归失败、绕过检查”等问题频发 [6]。
- 在漏洞修复验证上，智能体虽已展现出与专用系统相当甚至更优的补丁存在性测试（PPT）性能 [7]，但**补丁定位（patch localization）**仍是主要瓶颈，且对”**演化补丁（patch evolution）**”的鲁棒性尚未解决——这部分能力本质上是漏洞语义理解问题。

由此可见，三个环节在智能体范式下并非彼此孤立，而是被一条隐含的主线紧密连接：**对漏洞—补丁—修复三者语义的统一建模与跨环节复用**。抓住这条主线，就有可能突破现有“单点智能体应用”的局限，构建面向完整攻防闭环的智能体安全分析体系。

1.1.4 选题的提出

基于上述观察，本文提出“**基于智能体的漏洞挖掘与修复**”这一研究课题，以“漏洞语义—补丁语义—修复语义”的统一建模为核心，系统研究智能体在语义漏洞挖掘、漏洞自动修复、漏洞修复验证三个关键环节中的能力边界、瓶颈机理与增强方法，并探索三个环节之间的能力迁移与反馈闭环。

1.2 选题意义

1.2.1 理论意义

1. **推动“智能体 + 程序分析”交叉范式的形成**：现有智能体安全研究多停留在“把 LLM 当作更强的分类器/生成器”层面，缺乏对智能体推理过程与程序分析约束之间关系的理论刻画。本课题将探索智能体的“观察—推理—行动”循环与污点分析、符号执行、规约验证等传统程序分析技术的耦合机制，为构建新型混合分析范式提供理论支撑。
2. **建立漏洞语义跨环节统一表示**：针对漏洞挖掘、修复、修复验证三环节语义割裂的现状，本课题尝试建立以“漏洞触发条件—补丁干预点—修复不变式”为核心的统一表示，为后续跨任务迁移学习、多任务智能体训练提供概念框架。
3. **深化对“自因变量”与“演化补丁”两类共性瓶颈的理解**：前者是语义漏洞挖掘的核心障碍，后者是 PPT 与修复共同面对的难点。本课题将从智能体视角对二者进行系统建模，形成可被后续研究复用的分析框架。

1.2.2 实践意义

1. **提升语义漏洞挖掘的自动化水平**：相比内存破坏漏洞，语义/逻辑漏洞（认证绕过、状态机错误、配置驱动缺陷）长期缺乏高效自动化检测手段。基

于智能体的挖掘方法有望通过自然语言规约理解、内部状态推断与测试用例合成，显著扩展可检测漏洞类型。

2. **提高自动修复的可信度**：通过将 PPT 思想引入修复验证环节，本课题旨在构建“修复—验证”闭环，减少“补丁过拟合 PoC”、“修复引入回归”等问题，提升智能体生成补丁的真实可用率，为软件供应链的快速应急响应提供工具支撑。
3. **支撑大规模供应链安全审计与修复状态核查**：PPT 是软件物料清单(SBOM)与漏洞影响评估的关键技术，也是修复验证能力的直接应用。基于智能体的端到端 PPT 可显著降低对符号表与人工目标函数定位的依赖，提升对剥离二进制与真实发行版包的分析能力，对关键基础设施软件资产盘点与修复状态核查具有直接价值。
4. **服务国家网络空间安全战略**：漏洞攻防能力是国家网络空间安全的核心竞争力之一。本课题围绕智能体这一新兴技术形态，系统研究其在漏洞攻防关键环节中的应用边界与增强路径，可为我国在下一代自动化攻防体系建设中抢占先机提供理论与技术储备。

二 国内外本学科领域的发展现状与趋势

本章围绕课题的三个研究点，分别梳理相关领域的研究背景、研究现状与趋势、以及尚存的关键挑战。其中，2.1 节对应语义漏洞挖掘研究点，2.2 节与 2.3 节分别对应拟开展的漏洞自动修复与漏洞修复验证研究。

2.1 基于智能体的语义漏洞挖掘

2.1.1 研究背景

漏洞挖掘旨在识别目标系统中违反安全要求的程序行为，并提供支持缺陷判定与复现的证据。围绕这一目标，静态分析、符号执行和模糊测试分别从程序结构、路径约束和实际执行出发开展检测，并形成相互补充的技术体系。其中，覆盖率引导的模糊测试通过持续生成输入、执行程序并保留具有探索价值的测试用例，显著提高了自动化发现缺陷的效率；AddressSanitizer 等运行时检测机制

则为越界访问、释放后使用等错误提供了可复用的判据，使部分内存安全问题能够在执行过程中被直接识别 [8, 9]。这一体系的有效运行，不仅依赖输入生成能力，还依赖目标可执行性、关键状态可达性以及错误是否具有可观察的判定依据。

大语言模型和智能体的发展正在降低代码理解、测试程序编写及分析工具编排的人工成本。Google 的 Big Sleep 已通过代码分析与工具交互，在经过长期测试的 SQLite 中发现未知内存安全漏洞；Mozilla 于 2026 年 4 月报告，在使用前沿模型开展的初步评估中发现的 271 项漏洞已随 Firefox 150 修复 [2, 10]。这些进展说明，智能体已能够参与成熟项目的实际安全分析，但也表明内存安全问题并未被普遍解决。更准确的趋势是：**对于源码可用、执行环境完备、错误判据明确的部分任务，自动化能力已明显增强；在长期充分测试的特定组件中，仅继续增加同类输入变异的边际收益可能下降，新的增量能力更加依赖对深层程序语义与测试盲区的理解。**因此，后续研究不宜仅以“用更强模型生成更多输入、发现更多崩溃”为目标，而需要考察自动化能力尚未充分覆盖的任务条件。

与具有直接运行时告警的错误相比，协议状态违例、语义实现错误和资源生命周期缺陷往往缺少统一的判定信号。程序可能正常返回，却错误地保留会话权限、混用不同请求的状态，或未履行资源释放义务。此时，增加测试执行次数并不能自动解决“如何知道结果是错的”这一测试预言（test oracle）问题 [11]。本节将**语义漏洞**作为工作性概念，重点指需要结合协议规约、接口契约、状态关系或资源管理要求才能判定的安全缺陷；将 **silent bug** 用于描述在常规测试观察下不立即表现为崩溃或显式错误的缺陷。两者并非互斥于内存安全问题的分类：状态错误可能进一步引起非法访问，资源泄漏也可能在持续运行后才造成服务失效。普通功能错误与安全漏洞同样需要区分，只有能够联系到保密性、完整性、授权边界或可用性要求的缺陷，才构成本课题的安全研究对象 [11, 12]。

上述问题在嵌入式与固件场景中进一步受到分析条件的限制。仅能与设备交互时，内部状态、代码覆盖及资源对象通常不可直接观察；获得剥离符号的二进制后，虽然能够开展反汇编分析，但仍需恢复对象、功能与程序位置之间的对应关系；即使固件能够重托管，其行为也可能依赖外设响应、配置、定时事件与跨组件交互。Firmalice 和 Fuzzware 分别从二进制安全语义分析与硬件接口建模出发处理其中的部分困难，说明这些限制需要专门的程序分析与执行支持，不能

直接由源码级工具能力推导解决 [12, 13]。

因此，本节关注的问题并不是一般意义上智能体“能否发现漏洞”，而是：**在源码、符号、规约或运行时观察不完备的条件下，如何利用模型的语义理解与交互分析能力，发现违反行为约束且能够在真实环境中成立的安全缺陷**。研究重点由单纯增加输入与路径覆盖，进一步转向安全要求的获得、目标行为的辨识、状态违例的触发及其可信判定，为后续研究内容和技术路线提供问题依据。

2.1.2 研究现状与趋势

(一) 传统漏洞挖掘：从路径覆盖到状态与性质引导 传统漏洞挖掘并非只围绕崩溃展开。静态分析和符号执行能够在明确安全模型的前提下检查危险行为，Firmalice 即通过二进制切片与符号执行，分析攻击者是否能够获得执行特权操作所需的输入，以检测认证绕过问题 [12]。该工作说明，逻辑安全条件可以成为自动化分析的对象；其关键前提则是首先确定特权操作、攻击者能力与认证要求。因而，这一路线的困难不只是探索程序路径，还包括如何为不同系统建立具有实际安全含义的分析目标。

对于协议和有状态服务，Pham 等提出的 **AFLNet** 将消息序列而非单次输入作为测试对象，结合响应状态与代码覆盖进行探索 [14]。Natella 提出的 **StateAFL** 进一步通过长期存活内存的快照推断服务状态，降低对特定协议响应码的依赖 [15]。这些方法改善了多步交互中的状态探索，但其反馈主要描述被测实现已经出现了哪些状态，并不自动给出这些状态是否符合规范的判断。“**观察到新的实现状态**”与“**发现不应出现的安全状态**”是不同的目标：同一条代码路径可能在不同会话、权限或资源历史下产生不同安全结果，而代码覆盖本身不包含这一判断依据。

性质引导测试则将显式行为要求引入输入搜索。Meng 等提出的 **LTL-Fuzzer** 以线性时序逻辑性质为输入，通过灰盒模糊测试寻找违反事件时序要求的执行轨迹，而不是只接近某个代码位置 [16]。Yao 等提出的 **CFPOFuzz** 进一步面向上下文无关性质，将下推自动机监测状态与模糊测试调度结合，以处理具有嵌套或配对关系的行为约束 [17]。这类研究已建立“性质—轨迹—反例”的有效联系，但通常需要预先给出性质及其程序事件映射。因此，仍需进一步解决的并非是否可以按性质开展测试，而是如何从不完整材料中获得可信的性质，并将其准确落

实到缺少语义信息的目标程序或设备行为上。

(二) 大模型增强漏洞挖掘：从输入生成到交互式分析 大模型较早在漏洞挖掘流程中承担输入和测试程序生成任务。Xia 等提出的 **Fuzz4All** 将模型作为输入生成与变异引擎，通过自动提示构建支持不同输入语言和特性的测试 [18]。Meng 等提出的 **ChatAFL** 则利用模型的协议知识构造消息语法、丰富种子，并在覆盖停滞时生成可能到达新状态的后续消息 [19]。这些方法降低了手工编写生成器和补充协议输入知识的成本，主要增强“如何形成有效测试、如何进入更多状态”的能力；是否已经违反安全要求，仍取决于实际采用的错误检测机制。由此，输入生成能力与错误判定能力需要分别评价。

在系统语言场景中，Androutsopoulos 和 Bianchi 提出的 **deepSURF** 将静态分析与 LLM 增强的模糊测试驱动结合，针对 Rust 库中的 `unsafe` 实现生成测试。该方法不仅生成 API 调用，还处理泛型替换及自定义 trait 实现，使测试能够表达使用者行为，并通过更复杂的 API 交互触及底层缺陷 [20]。其价值在于扩大能够有效构造和执行的客户端行为范围，而不只是修改输入字节。同时，**deepSURF** 的检测目标仍是内存安全问题，不能将其 API 组合能力直接等同于协议、授权或资源性质的通用判定能力。

进一步的智能体化探索将模型置于持续的工具交互过程。**Big Sleep** 利用代码浏览、调试和测试等工具调查候选问题，其早期 SQLite 案例还结合了近期代码变化带来的分析线索 [2]。这表明模型能够参与假设提出、证据获取与执行验证，而不局限于单次代码分类。综合上述进展，LLM 与传统分析的结合正从某个固定环节的辅助生成，扩展到对整个分析过程的组织；但智能体能否自主操作工具，与其提出的安全假设是否正确、测试是否真正触发目标行为，仍是需要分别回答的问题。

(三) 二进制与固件分析：增强语义恢复、执行支持和证据获取 对于剥离符号的程序，Liu 等提出的 **BINREX** 通过语义检索与可检查的分析操作支持任务驱动的二进制分析。其检索模块基于微调的 Pythia-1B 模型建立二进制函数与文本或函数名之间的语义联系，并通过分层规划、受约束的中间表示及 IDAPython 代码生成组织证据获取 [21]。这一设计缓解了模型在大量无语义函数中定位分析目标、正确使用二进制工具的困难。不过，论文明确将研究范围限定为能够从代

码与数据推导的静态分析任务，将根本依赖动态执行的竞争条件、时序和环境相关触发排除在当前范围之外。因此，二进制语义检索与工具执行的进步是状态性漏洞研究的基础，而非对其可达性和行为正确性的完整解决。

Ji 等提出的 **FirmAgent** 利用模糊测试收集实际运行中的输入相关代码点，再由 LLM 智能体开展上下文感知的污点分析，并根据已有测试构造漏洞见证 [22]。Jia 等提出的 **PANGOLIN** 则面向多语言 IoT Web 服务，利用模型识别请求分派机制与隐藏接口，通过跨语言分析恢复分层参数约束，并根据响应反馈修正输入规格 [23]。两者分别增强了输入来源与危险路径之间的联系，以及接口与参数语义的恢复。需要注意，PANGOLIN 对真实设备开展测试并不意味着其只依赖黑盒设备信息，其分析仍利用固件实现；输入参数规格也主要说明“接口如何调用”，不能替代“接口在不同状态下必须满足什么安全要求”的行为规约。

执行环境方面，**Fuzzware** 通过分析固件实际如何使用 MMIO 返回值，构建有助于缩小输入空间的硬件接口模型 [13]。Wang 等提出的 **FIRMENSTEIN** 则以 API 依赖图支持跨程序分析，由智能体诊断 Linux 固件服务的重托管障碍，并合成用于处理环境依赖的干预代码 [24]。这些研究提高了固件可执行性与动态分析覆盖，为后续测试提供了必要条件。但从研究目标上看，“服务能够运行和响应”不同于“所有安全相关状态都与真实设备一致”；对依赖硬件或环境语义的候选漏洞，仍需解释环境模型的适用范围，以及发现是否依赖额外干预。

Peng 等提出的 **Bond** 从已有静态污点报告出发，通过约束引导的定向模糊测试验证 Linux IoT 固件中的候选路径，并生成可复现证据 [25]。这进一步说明，报告与实际触发之间需要专门的验证环节。总体上，现有研究已分别改善语义定位、输入恢复、环境构造和候选路径验证，但这些能力并不自动组成面向未知语义漏洞的完整机制：其中一些方法以已有危险操作或污点报告为目标，另一些方法优化可执行性或输入覆盖，尚不能据此推断其已解决无现成 oracle 时的状态违例识别。

(四) 协议语义分析：从报文有效性走向实现合规性 针对协议逻辑错误，Yang 等提出的 **BSFuzzer** 直接以 Bluetooth Core Specification 为依据，利用 LLM 智能体从文本、图示及上下文中恢复状态机与报文语义，构造字段规则违例和状态转移扰动，并在真实 BLE 设备上检查交互响应 [26]。与主要提升输入语法有效性

和状态覆盖的方法相比，BSFuzzer 将关注点推进到协议实现是否正确处理特定违例，表明模型可以参与从规约理解到语义测试的过程。这一结果为设备黑盒条件下的逻辑漏洞发现提供了直接依据，而不应被简化为仅适用于有源码场景的探索。

Song 等提出的 **ProtocolGuard** 沿另一条路线，将规约与源码实现建立联系：从协议规范提取约束，利用 LLM 与 LLVM 分析获得规则相关代码切片，识别实现不一致，再生成断言并通过定向模糊测试验证 [27]。其重要进展是将原本没有明显错误信号的行为转化为可执行断言，而非仅输出模型认为可疑的代码。因此，“从规范提取规则并驱动测试”已经有实质性研究基础，新的研究不能只重复这一流程名称。

然而，已有方法的适用条件和未解决问题仍然明确。BSFuzzer 依赖较完整的 Bluetooth 规范和协议交互信息，论文指出涉及历史状态与多步依赖时，LLM 响应验证仍可能误判或漏判 [26]。ProtocolGuard 当前面向可分析与插桩的 C 源码，其讨论指出，异步回调和脱离消息处理调用链的清理逻辑可能导致切片不完整，断言生成与动态触发也会受到上下文和输入可达性的限制 [27]。因此，进一步的研究空间在于：当完整规约、源码变量和插桩条件不能同时获得时，如何建立足够可信的状态与行为证据，并判断目标对异常交互的处理是否真正构成安全违例。特别是，**测试端发送非法报文或打乱消息顺序，只是施加测试刺激；目标未按要求拒绝、隔离或恢复，才可能构成实现缺陷。**

(五) 类型状态与历史补丁：从缺陷经验提取行为约束 类型状态 (typestate) 将对象允许执行的操作与其所处状态联系起来，为描述资源生命周期和接口调用顺序提供了方法基础。Potyondy 等提出的 **Abacus** 面向硬件驱动正确性，通过稳定状态 (stable state) 与瞬态状态 (transient state) 区分硬件—软件交互中的状态约束：某些状态只能由软件操作改变，另一些状态则可能由硬件独立推进 [28]。这一划分说明，仅根据软件 API 调用顺序描述嵌入式状态并不总是充分，还必须考虑硬件行为与软件观察之间的关系。不过，Abacus 是基于 Rust 类型机制的驱动构造与约束框架，其协议标注由开发者提供，并非从现有二进制自动恢复规约或生成漏洞测试的系统。

Wang 等提出的 **TyPatch** 预印本则从历史 Linux 补丁中提取可复用的缺陷知

识。该方法由 LLM 生成包含跟踪对象、动作、守卫条件、状态转移和违例的 `typestate` 规则，再交由共享静态分析后端处理对象身份、别名和跨过程传播，避免要求模型为每条补丁重新实现完整分析器 [29]。其结果展示了历史修复向检测规则转化的价值，但输出静态规则与获得目标环境中的可执行违例轨迹仍属于不同任务。历史补丁表达的约束也可能依赖原项目的接口和错误处理约定，需要在新目标上重新检查其适用条件。

结合 CFPOFuzz 对更丰富时序性质的探索可以看出，状态规则为缺陷知识复用和测试反馈提供了有价值的接口，但不意味着所有目标都适合被压缩为简单有限状态机 [17, 28, 29]。对象配对、嵌套操作、资源计数和硬件异步变化可能要求不同的抽象粒度。因此，值得进一步研究的是行为约束在不同实现中的可信获得与可执行解释，而不仅是从文档中生成一张状态图。

(六) 非崩溃错误与资源缺陷：扩展输入之外的判错能力 对于没有直接崩溃信号的功能错误，Yang 等提出的 **LISA** 联合生成 API 调用序列与程序不变式，并利用 API n-gram 反馈引导测试 [30]。这使测试不仅覆盖不同调用组合，还尝试判断组合执行后的行为是否满足预期要求。论文将结果定位为供开发者确认的高置信候选，体现出自动生成不变式与确认其正确性之间仍有距离。该研究面向软件库功能缺陷，其思想可以为安全测试提供借鉴，但其中的功能不一致不能未经安全影响分析就全部计为漏洞。

Zhang 等提出的 **TransFuzz** 面向深度学习库的 silent bug，从历史问题中提取带上下文的缺陷模式，匹配功能相关 API，再合成包含定制 oracle 的测试，并通过自验证约束迁移过程 [31]。与仅生成输入相比，它将历史案例中“何种行为意味着错误”的知识也纳入迁移，说明模型能够同时帮助构造刺激与判据。其已验证能力主要位于具有源码、API 与测试环境的深度学习库；向固件迁移时仍需重新解决行为约束的适用性、内部状态的可观察性和实际交互入口问题，不能直接沿用该领域的测试假设。

资源缺陷同样具有传统方法基础。Wen 等提出的 **MemLock** 将内存消耗引入模糊测试反馈，以发现不受控资源使用问题 [32]；Wang 等提出的 **InferROI** 利用 LLM 推断资源获取、释放与可达性验证意图，再结合静态控制流分析检测资源泄漏 [33]。前者表明资源使用能够成为输入搜索目标，后者则说明资源操作的语

义不必完全依赖预定义 API 名单。与此同时，LeakSanitizer 已能够检测其支持环境中的堆内存泄漏 [34]，因此不能把资源泄漏整体描述为“已有工具无法检测”的类别。

更具挑战性的是具有特定生命周期要求的长期资源占用：例如，请求取消或连接异常结束之后，仍可通过容器访问的对象可能已经失去业务用途；占用增长也可能来自合法缓存或延迟回收，而非泄漏。由这些情况可见，单独观察内存曲线，或只判断对象是否可达，都不足以决定它是否违反了释放义务。嵌入式目标还可能涉及句柄、连接槽、任务对象等资源，其检查需要同时解释资源所属对象、应当结束的事件和允许的回收条件。InferROI 提供了资源意图建模的基础，但其源码级静态分析能力不能直接推广为设备黑盒条件下的长期资源行为判定 [33, 34]。

综合以上研究，漏洞挖掘正在由“更多有效输入”向“更有依据的行为探索”发展。现有工作已经分别展示了规约提取、状态建模、规则执行、定制 oracle 与二进制智能分析的价值，尚需进一步打通的关键联系是：**在受限信息与执行条件下，将安全要求映射为目标可观察的行为条件，再形成真实可达、可复现且能够排除错误规约解释的违例证据**。这一问题既不同于单纯提升反编译理解，也不同于在已有完整 oracle 的条件下优化模糊测试速度。

2.1.3 研究挑战

(一) 不完整与多来源证据下，行为规约的可信获得 协议文档、接口说明、源码和历史补丁提供的不是同一种证据。规范描述应有行为，实现展示当前行为，补丁反映特定版本中需要改变的条件；三者可能存在省略、歧义或适用范围差异。ProtocolGuard 对规则上下文的处理和 Abacus 对开发者标注正确性的依赖表明，规约来源及解释错误会直接影响后续检测 [27, 28]。因此，**不能将从待测实现中归纳的常见行为直接作为正确规范，也不能仅凭模型生成了自洽解释就确认安全要求成立**。

进一步的难点是，在缺少完整规范时仍然保留检测能力，同时说明结论所依赖的前置条件和不确定性。历史缺陷可以提供检查方向，但目标程序的版本、对象类型和环境可能改变规则含义。需要区分明确规定的安全义务、具有独立证据支持的推断以及尚待验证的候选假设，并据此界定误报与漏报，而不是为了获得

更多告警而放宽规约可信度 [11, 29]。

(二) 缺少源码与符号时，安全概念与目标行为的准确对应 BINREX 已增强了剥离二进制中的语义检索与静态证据获取，但状态性缺陷要求建立的联系通常更具体：某段代码操作的是哪个会话或资源，某次状态写入影响哪些后续行为，以及是否存在跨函数、跨任务或跨组件的共同维护关系 [21, 23]。仅识别出“认证函数”或“资源释放函数”，不足以确认特定权限或生命周期约束是否被违反。

设备黑盒条件进一步限制了这一映射。相同响应可能对应不同内部状态，响应变化也可能来自超时、丢包或实现允许的可选行为。BSFuzzer 在复杂历史状态下的验证局限说明，交互日志需要与适用规则及上下文一起解释 [26]。因此，必须区分源码可用、二进制可用与仅设备可用等条件，研究在不同观察能力下能够成立的判定，而不能默认所有目标均可获取覆盖率、内部变量或资源计数。

(三) 跨事件状态与环境依赖下，违例轨迹的真实可达性 深层语义缺陷往往依赖先前调用、异常路径和环境事件共同建立的状态。AFLNet、StateAFL 与 ChatAFL 已将多步交互纳入搜索，Abacus 则指出硬件状态变化不能总由软件操作序列完整刻画 [14, 15, 19, 28]。因此，测试既需要维持进入目标状态所需的有效交互，又要在关键条件上施加扰动；无约束地生成非法操作可能只得到早期拒绝，而无法触及真正需要检查的行为。

在重托管或局部执行中，还需要区分“可以人为构造某个内部状态”与“该状态能够由允许的外部操作到达”。模拟返回值、补齐环境和复位设备可以辅助测试，但不能将任意修改认证结果或内部对象状态所得到的异常，直接作为真实攻击条件下的漏洞。Fuzzware、FIRMENSTEIN 与 Bond 分别提供环境建模和动态验证的基础，进一步的挑战是将这些能力与目标安全性质联系起来，说明违例是否依赖不成立的环境假设，并提供可重复的触发过程 [13, 24, 25]。

(四) 无直接错误信号时，语义 oracle 的有效性与区分能力 LISA、TransFuzz 和 ProtocolGuard 已开始自动构造不变式、定制 oracle 或断言，但生成一个能够执行的检查，不等于该检查具有正确的安全含义 [27, 30, 31]。判据过弱可能遗漏缺陷，过强可能拒绝规范允许的行为；若测试生成与结果判定共享同一个错误假

设，完整闭环也可能稳定地产生误报。差分结果只能说明实现之间存在不一致，仍需进一步确定差异是否允许、哪个结果有误及其安全影响 [11, 27]。

对资源泄漏和延迟显现的错误，问题还涉及观察窗口与生命周期边界。有限时间内没有释放，可能是缺陷，也可能是合法的异步回收；资源持续增长可能反映无界保留，也可能是尚未稳定的缓存。逐例重启服务可能清除跨请求积累的资源状态，而持续复用进程又可能引入前序测试残留，增加错误归因的困难。因而，需要对资源使用的预期边界、结束条件与合法对照行为给出明确解释，并将候选发现与独立确认分开。研究目标应是获得可反驳、可复现的安全违例证据，而不是用一个模型的判断替代原先的崩溃判据 [32–34]。

(五) 从单点能力增强到可评价的智能体发现过程 现有工作的反馈目标各不相同：代码与状态覆盖衡量探索进度，性质监测状态反映特定行为约束的接近程度，资源消耗则提供另一类风险信号 [14–17, 32]。这些信号不能简单互相替代，也不宜无条件累加。例如，一个不再增加代码覆盖的调用序列，仍可能改变同一对象的生命周期历史；但不断制造新的模型状态标签，也不代表发现了新的安全行为。如何在有限预算下利用不同反馈，并以实际漏洞证据而非代理指标检验效果，是智能体化测试需要回答的问题。

评价还应保持输入条件与证据标准一致：提供完整源码和手工规约的任务，与只给固件镜像或设备的任务不能直接比较；静态候选、断言失败、可复现缺陷和经确认的安全漏洞也应分别记录。对于利用历史案例的系统，需要避免将目标漏洞的未来修复泄漏为发现先验；对于交互式系统，应保留环境构建、工具失败、推理、测试和人工复核的完整成本。只有明确这些边界，才能判断收益来自模型能力、分析方法还是额外信息，而不是将有利设置下的成功率推广到所有嵌入式场景 [21, 25, 29–31]。

综上，智能体与大模型已经显著扩展自动漏洞挖掘的能力，但能力提升并未消除规约缺失、状态不可见和非崩溃行为难以判定等问题。面向嵌入式设备和剥离二进制等受限场景，有必要在已有规约分析和性质引导测试的基础上，进一步研究**可信行为约束、真实可达交互与可靠语义判定之间的联系**，重点覆盖协议违例、跨调用语义错误及资源生命周期缺陷。在固件场景中，这类由内部状态、配置、时序与外设共同决定程序行为的变量可统一视为**自因变量**（其形式化

定义见 3.2 节), 是连接行为规约、可达性分析与语义判定的枢纽概念。这一问题定位为本课题进一步研究语义漏洞挖掘方法提供了依据。

2.2 基于智能体的漏洞自动修复

2.2.1 研究背景

漏洞修复是软件安全维护的重要环节, 其目标不仅是阻止已知攻击输入触发异常, 还需要消除导致危险行为的程序缺陷, 并维持正常使用所依赖的功能与安全约定。漏洞自动修复 (Automated Vulnerability Repair, AVR) 将漏洞分析、补丁生成与补丁验证组织为自动化过程, 以降低安全维护对人工分析的依赖。与面向一般软件缺陷的自动程序修复 (Automated Program Repair, APR) 相比, AVR 更强调在攻击者可控输入、异常状态及安全敏感操作下恢复必要的安全条件。因此, 修复目标不能仅用代码修改是否合理、程序能否编译或已知失败是否消失来描述, 还应考察危险行为是否被有效约束, 以及相关合法行为是否得到保留 [35]。

漏洞修复通常发生在不完备信息条件下。漏洞报告、检测器告警和概念验证程序 (Proof of Concept, PoC) 能够提供分析线索, 但不一定直接指出应修改的程序位置, 也不一定完整描述正确修复应满足的条件。已有 APR 研究表明, 通过用于生成补丁的测试集, 并不意味着补丁能够推广到未见输入 [36]。在漏洞修复中, 这一问题进一步表现为: 已知 PoC 不再触发, 可能只是因为某个特定输入或执行分支被阻断, 而非漏洞根因已经消除。由此, AVR 实际上包含两个相互关联、但不能相互替代的问题: **如何根据有限证据推断应当修复什么, 以及如何验证生成的补丁确实满足修复要求** [35, 36]。

大语言模型 (Large Language Model, LLM) 为处理漏洞描述、源代码和修复经验提供了新的技术途径。进一步结合代码检索、文件编辑、编译执行与测试工具, 模型能够在智能体框架中逐步完成定位、修改和反馈驱动的修复。PatchAgent 等工作已经将这些环节整合到交互式系统中, 推动研究对象从给定代码片段上的补丁生成扩展至真实项目中的自动修复 [37]。然而, 交互能力的增强并不自动等同于修复语义的正确理解: 系统仍需判断告警位置与根因位置之间的关系、识别跨函数依赖, 并区分补丁通过当前验证与恢复完整安全要求之间的差别。

Rust 等系统编程语言使这一问题具有更丰富的语义层次。RustBelt 从形式化角度说明, 基于 `unsafe` 实现的安全抽象需要满足相应验证条件, 才能作为安全

语言的可靠扩展 [38]；针对真实 Rust 程序的研究也揭示了内存与线程安全实践中的复杂问题 [39]。因此，Rust 漏洞修复不仅涉及局部条件检查，还可能涉及所有权、生命周期、类型与 trait 约束、异常清理和并发状态。基于上述背景，本节围绕传统修复方法、LLM 与智能体修复、补丁正确性评价及 Rust 修复进展展开，重点讨论从漏洞表现恢复根因语义、从候选修改建立可信修复证据所面临的挑战。

2.2.2 研究现状与趋势

(一) 基于搜索、程序合成与语义约束的传统修复 传统 APR 为漏洞自动修复提供了候选搜索、修复位置选择和测试反馈等基础机制。Le Goues 等提出的 **GenProg** 通过遗传编程对程序进行变换，以失败测试与功能测试约束候选程序的选择 [40]。Nguyen 等提出的 **SemFix** 将通过给定测试的要求编码为约束，并结合符号执行、约束求解和程序合成搜索修复表达式 [41]。二者分别代表基于搜索和基于语义约束的修复思路，但主要面向一般程序缺陷，不能直接作为已覆盖多类安全漏洞的证据；其生成结果的可靠性还取决于所使用的测试、约束及允许的程序变换。

面向安全缺陷，研究进一步尝试从漏洞执行中提取比单次测试结果更强的修复条件。Gao 等提出的 **ExtractFix** 从崩溃或漏洞触发中提取约束，通过程序分析将其向候选修复位置传播，再合成满足约束的修改 [42]。该方法不只要求原始输入不再崩溃，而是将崩溃所违反的条件转化为修复义务，探索对输入变化更具一般性的修复。Zhang 等提出的 **VulnFix** 则利用程序状态快照、状态变异与归纳推断学习区分安全和危险状态的不变式，并据此生成补丁 [43]。这些研究说明，安全条件建模和反例反馈并非 LLM 出现后才引入的思想，传统语义方法已经为约束驱动修复提供了基础。

不过，这类方法的分析与合成建立在具体的漏洞模型、状态表示和候选修复空间之上。对于缺乏通用崩溃判据的权限、协议和状态逻辑问题，或者依赖复杂库语义的跨过程缺陷，单一崩溃约束难以完整表达修复要求；大型项目的路径、别名与环境建模也会增加分析成本 [35, 42, 43]。因此，传统方法的局限不宜简单归结为“缺乏语义分析”，而应理解为：已有语义表示与求解机制如何适应更复杂的安全要求，以及如何在精度与可扩展性之间取得平衡。

(二) **从历史补丁学习到大语言模型生成** 学习式 AVR 将漏洞代码到修复代码的映射作为建模对象，从历史修复数据中获得候选生成能力。Fu 等提出的 **VulRepair** 基于 T5 类编码器—解码器架构，结合预训练与子词表示学习漏洞修复模式 [44]。Zhou 等提出的 **VulMaster** 进一步扩展输入范围和信息来源，缓解代码截断及修复上下文不足的问题 [45]。这类方法降低了逐类手工设计补丁模板的需求，但仍需考虑训练数据覆盖、输入信息完整性以及评价方式的影响；与参考补丁相似，并不直接等同于恢复了相应安全性质。

随着通用代码模型的发展，研究开始探索不进行任务专门训练的修复方式。Pearce 等系统考察了**零样本漏洞修复**，通过组织漏洞代码与提示信息引导模型生成候选修改，展示了预训练模型迁移至安全修复任务的可能性 [46]。与面向固定数据分布的训练方式相比，提示式方法便于接入不同模型和额外上下文；但模型输出仍需接受程序级检查，提示中缺失的调用关系、错误处理约定或边界条件，不会因为模型具有代码生成能力而自动得到可靠补全。

训练式方法也在从修复结果拟合转向显式推理能力学习。Wen 等提出的 **VulR2** 结合领域推理数据、监督微调与课程式奖励训练，增强模型对漏洞成因及修复过程的建模 [47]。需要注意，其训练中的可验证奖励包含问答与字符级匹配等信号，并不等同于对完整仓库执行安全性质验证。总体而言，训练式与提示式方法是互补路线，关键问题不是预设某一路径必然更优，而是判断所获得的推理和生成能力能否转化为可应用、可执行且安全的补丁。

(三) **上下文增强、根因分析与经验引导修复** 直接向模型提供漏洞函数并要求修改，容易遗漏决定修复策略的跨函数关系与领域约定。针对这一问题，Nong 等提出的 **APPATCH** 通过静态依赖分析提取漏洞相关代码，结合根因分析、自适应示例选择和渐进式提示生成补丁 [48]。其根因分析能够根据模型提出的信息需求逐步扩展相关函数切片；示例选择也不只依赖代码相似性，而是比较漏洞成因，以引导后续修复。该工作已经将跨过程上下文、根因推理和历史经验整合起来，并非仅在固定函数文本上进行一次性生成。

APPATCH 同时显示了这一技术路线的边界：其主要流程以漏洞相关位置及类别为分析起点，论文也进一步考察了与 CodeQL 告警结合的端到端设置，因此不能将其概括为完全不能处理检测器输出。其失败分析仍涉及根因判断错误、上

下文不足和边界处理不完整等问题 [48]。独立的 **InterPVD** 研究则明确区分漏洞触发语句与待修复语句，并指出二者可能位于不同函数 [49]。这意味着”定位到漏洞表现”与”识别能够恢复安全条件的修改位置”需要分别评价。

从这些研究可以归纳出两个进一步的问题。其一，上下文扩展既要补齐关键定义、调用关系和状态依赖，又要避免将大量无关实现交给模型；仅扩大输入长度不能替代有针对性的证据组织。其二，若早期根因解释存在错误，以该解释选择示例和生成补丁可能继续强化同一假设。因而，根因分析不仅需要给出自然语言说明，还需要能够对照源码与执行证据检验其成立条件，并在证据冲突时修正解释。这是对上下文增强与因果示例选择流程的进一步研究要求，而不是否认已有方法已经开展根因分析 [48, 49]。

(四) 面向真实仓库的交互式修复智能体 与将 LLM 作为局部生成模块的流水线不同，修复智能体通过持续观察项目、选择工具和利用执行反馈组织分析过程。**SWE-agent** 研究智能体—计算机接口如何支持仓库导航、文件编辑和程序执行，表明工具接口设计会影响模型完成软件工程任务的方式 [50]。**RepairAgent** 则面向程序修复，使模型在信息收集、修复素材检索、候选修改和测试之间自主选择操作 [51]。这些工作主要提供通用软件工程或 APR 能力，不能仅凭通用基准上的任务完成率推断其在安全漏洞修复中的正确性。

PatchAgent 面向真实漏洞修复，将定位、补丁生成与验证整合到同一智能体中，并通过语言服务器及验证接口提供代码导航和反馈 [37]。这种设计使系统不必完全依赖预先固定的人工修复路径，能够根据构建失败、漏洞复现和回归测试调整候选补丁。不过，智能体能否持续执行修复，与反馈能否区分根因修复和表现抑制，是两个不同问题。多轮迭代可以提升对现有测试的适配程度，却不能单独保证测试未覆盖的行为也得到正确处理。

2026 年公开的 **Kumushi** 进一步将根因定位作为修复系统的核心，通过多样化动态定位与证据加权排序，为模型筛选更有诊断价值的程序位置；其评价同时结合自动验证与结构化专家审查，以区分通过 oracle 的补丁与更充分的根因修复 [52]。这一进展表明，研究重点已不仅是增加智能体的自主操作范围，而是改善”程序证据—根因判断—修复决策”之间的联系。与此同时，自动执行、人工审查和形式化证明具有不同的证据强度，不能因系统包含分析工具或多个角色，

就将其结果视为已获得充分正确性保证。

(五) 从可复现修复任务到独立补丁验证 修复方法的发展依赖能够复现漏洞及验证修改的实验基础。**Vul4J** 提供带有漏洞证明测试的 Java 漏洞案例，支持在真实项目中研究自动修复 [53]；Wu 等通过 **VJBench** 及相关实验考察神经网络修复安全漏洞的能力，为区分一般缺陷与安全修复需求提供了数据基础 [54]。**SEC-bench** 将任务扩展至真实 C/C++ 仓库中的漏洞复现与修复，并通过智能体自动完成相当部分案例构建工作 [55]。因此，现阶段的问题已经不只是缺少案例或依赖人工打包，而是如何在自动扩展基准的同时确保环境、漏洞证据和验证逻辑可靠。

补丁验证方面，Smith 等较早揭示了 APR 的测试过拟合问题 [36]。Yang 等提出的 **OPAD** 利用自动生成的测试输入与运行时安全判据，增强对过拟合补丁的识别 [56]。其意义在于，将用于产生补丁的有限测试之外的行为纳入检查；但增加输入数量主要扩展被执行的行为范围，能否发现错误仍取决于输入是否触及遗漏路径，以及 oracle 是否能够识别相应违规。针对权限策略、资源约束或接口契约等问题，仅观察崩溃并不足以判定修复质量。

2026 年的 **PVBench** 研究进一步引入开发者随官方修复新增的 **PoC⁺ 测试**，比较原有功能测试与 PoC 所形成的基础验证，以及包含修复后行为要求的增强验证 [57]。其结果显示，在所评估系统和案例上，基础验证接受的部分补丁仍会违反新增测试所表达的要求。这说明修复评价需要关注漏洞相关的预期行为，而非仅判断已知触发是否消失。另一方面，**PoC⁺** 也可能包含实现策略、性能或工程惯例方面的要求，因此使用这类检查时，应区分漏洞必要安全条件与更广泛的补丁质量要求，避免把所有偏离官方实现的方案统一判为不安全。

同年公开的 **PatchBench** 则同时关注补丁记忆、表层修复和功能保持：通过漏洞移植与代码变换调整任务上下文，利用额外漏洞输入检查安全性，并在正常输入上对照参考修复后的程序输出及项目测试检查语义 [58]。该研究也指出，验证仍依赖有限输入，且真实修复时通常没有可直接用于比较的参考修复程序。由此可见，**更强的基准评估与修复过程可获得的反馈之间仍存在信息差**：评估者可以保留官方补丁和独立测试，而修复器必须在这些信息不可见时形成有效修改；二者的信息边界需要明确区分。

在上述背景下，本课题组前期构建的 **RRBench** [6] 针对 Rust 生态缺乏大规模真实漏洞修复基准的问题，构建了覆盖 115 个案例、95 个仓库的仓库级修复基准，其两阶段评估设计与本节讨论的方向直接对应：公开阶段向修复系统提供漏洞触发 PoC 与基础功能测试，私有阶段使用对修复系统不可见的安全性质检查（security oracles）与合法行为检查，验证补丁是否真正恢复应有安全性质；对五款代表性修复智能体的评估表明，通过公开验证的提交中有 23.4% 未能通过私有安全性质检查，且存在 14 个任何系统均无法正确修复的案例。这说明功能测试会系统性高估漏洞修复能力，补丁验证必须显式建模安全性质，也为研究内容二”生成—验证”协同机制的探索提供了直接的实证动机。

(六) Rust 修复及其语言相关验证机制 Rust 修复研究已经覆盖多种任务，但不同工作的目标不能混为一谈。**RustAssistant** 利用编译器诊断与 LLM 迭代修复编译错误 [59]；**RUPAIR** 面向缓冲区溢出的检测与修复 [60]；**RUSPATCH** 研究 Rust 应用的动态补丁部署，重点在于及时应用修复，而非从漏洞描述中自主合成补丁 [61]。这些工作分别回答编译可接受性、特定缺陷修复和补丁应用问题，不能据此推导出对任意 Rust 安全漏洞的端到端修复能力。

在仓库级任务上，Xiang 等提出 **Rust-SWE-bench** 和 **RustForger**，通过自动测试环境设置及基于 Rust 元编程的动态跟踪，增强问题复现与程序分析 [62]。该工作面向一般 Rust 软件问题，其语言相关支持有助于仓库级修复，但基准任务完成并不自动证明漏洞安全要求已经恢复。**AkiraRust** 则面向未定义行为修复，使用有限状态机协调多智能体的快慢推理、运行时语义检查与回退 [63]。这表明执行语义已经进入 Rust 智能修复流程，但未定义行为检查与协议、权限、资源等安全要求的验证仍需采用不同机制。

Rust 的分析工具为此提供了互补支持。**Miri** 能在执行中识别多类未定义行为，但其文档明确说明，通过某些测试不能保证安全抽象对任意合法客户端均成立 [64]。**Loom** 通过模型化并发操作探索线程交错，但需要被测程序及相关组件使用其可观察的替代接口，未纳入模型的操作不在其分析范围内 [65]。Rust 语言参考还明确指出，unsafe 不豁免未定义行为约束，非法值、引用和别名关系等条件可能在危险状态形成时就被违反 [66]。因此，修复验证需要依据具体性质组合编译期客户端检查、执行检查、并发测试和合法行为对照，而不是寻找一个适

用于所有 Rust 漏洞的统一崩溃判据。

综合上述研究，基于智能体的 AVR 呈现出三个相互关联的发展方向：从给定局部代码的补丁生成扩展至真实仓库中的交互式分析；从单纯学习修改模式转向结合上下文、根因和程序证据组织修复；从已有测试通过转向独立的安全行为与合法功能验证。三者的交汇点在于：模型不仅要生成修改，还需要明确修改依据、作用边界及其验证范围 [37]。

2.2.3 研究挑战

(一) 不完备定位条件下，根因位置与修复上下文的可靠恢复 InterPVD 揭示了触发位置与待修复位置之间的跨过程差异，APPATCH 和 Kumushi 则分别从静态上下文与动态证据出发支持根因分析 [48, 49, 52]。然而，对于检测器仅提供崩溃位置、污点终点或局部告警的情况，系统仍需要追踪状态的建立、传播和使用关系，确定真正应承担安全检查或状态维护的程序位置。同时，扩大上下文可能引入噪声，过度裁剪又可能丢失别名、辅助函数及异常路径。关键挑战是将不确定的定位线索转化为可验证的依赖证据，并以这些证据决定何时扩展、裁剪或重新定位，而不是默认上游告警已经解决修复定位问题。

(二) 根因推理、历史经验与候选策略之间的语义一致性 已有方法已通过因果示例、推理训练及证据排序改善补丁生成，但根因解释与目标程序之间仍可能存在偏差 [47, 48, 52]。同一 CWE 类别只能提供缺陷类型线索，不能唯一确定被约束的对象、异常来源或正确干预方式。后续研究需要让候选解释接受数据流、控制关系、接口前置条件及执行观察的检验，并在证据不足时保留竞争性假设。历史案例也应作为待核对的经验，而非直接套用的模板。与之对应，候选补丁的多样性应体现不同的约束建立方式或修复位置，而不仅是对同一修改反复采样；增加候选数量是否改善正确修复覆盖，需要在相同预算下单独验证。

(三) 跨接口、状态与语言机制的安全性质恢复 RustBelt、真实 Rust 安全问题研究及语言参考表明，安全要求可能跨越类型接口、内部实现和多次调用，而不只对应一条局部语句 [38, 39, 66]。例如，限制一次危险访问并不必然修复产生非法引用的接口，改变异常返回也不必然修正形成错误状态的转移。因此，修复需要确定安全性质应当在何处建立、由哪些操作共同维护，以及哪些调用、清理或并

发路径仍可能绕过约束。对于生命周期、trait 和接口契约问题，增强编译期约束也可能是有效修复方式；评价时必须区分危险客户端被有意拒绝与库本身或合法客户端意外构建失败，不能仅以运行时 PoC 的执行结果概括修复成败。

(四) 部分规约下，独立验证与合法行为保持的可执行化 OPAD、PVBench 和 PatchBench 已从额外输入、开发者新增测试和输出语义等角度增强验证，但这些机制仍受测试覆盖、参考信息及 oracle 适用范围限制 [56–58]。进一步的难点在于：从漏洞描述和程序证据中提取确实必要的安全条件，将其映射为能够执行的检查，并同时保留与危险行为相邻的合法用法。验证不仅要防止检查过弱而接受不完整修复，也要防止检查过强而拒绝不同于官方实现的正确方案。对于自动生成的验证逻辑，还需检查其前置条件、预期结果和程序映射，避免用模型生成的另一段解释为候选补丁自我背书。

这里还需要严格区分修复期反馈与最终评价依据。修复期可以利用公开测试和自行生成的反例迭代，但最终保留测试不应反复返回给修复器，否则其独立性会被削弱。对于 Rust，Miri、Loom 与编译期检查可以提供不同侧面的反例或支持证据，但均需要说明模型、输入和环境范围 [64, 65]。**通过有限验证表示满足已检查的要求，而不是证明程序在所有可能执行中均无漏洞。**这一边界同样意味着，核实某个修复是否存在的 PPT 不能单独替代补丁充分性与功能保持验证 [36]。

(五) 真实任务、评价标准与资源成本的统一刻画 不同修复研究在输入粒度、定位提示、运行环境和判定标准上存在明显差异：有的评价给定漏洞位置上的候选生成，有的要求系统自行分析仓库；有的采用参考补丁比较或人工审查，有的依赖可执行测试 [35, 48, 55, 57]。因此，需要在明确输入与信息权限的前提下，分别报告补丁生成、可应用构建、公开测试及独立验证的结果，并保留无补丁、环境失败和预算耗尽等情况。不能将不同证据标准下的“正确率”直接排序，也不能把基准任务的成功等同于已经完成生产环境中的安全修复。

此外，历史修复可能出现在训练或检索数据中，模型与官方补丁相似并不足以区分经验泛化和直接记忆；PatchBench 对任务上下文变换的研究进一步说明了控制这一风险的必要性 [58]。面向可复现研究，还应固定源版本、工具链、模型配置和检索边界，并在成本评价中纳入上下文获取、候选生成、测试与复杂验

证的完整开销。最终需要回答的不是某个模型在最有利设置下能够生成多少补丁，而是在有限预算和不完备证据条件下，系统能够多可靠地完成根因定位、生成有效修改并提供独立验证证据。

综上，基于智能体的漏洞自动修复已经具备从局部生成走向交互式仓库分析的技术基础，但可信修复仍依赖两个相互衔接的能力：一是从不完备定位与上下文中形成有证据支持的根因判断，并据此探索适当的修复策略；二是把漏洞相关安全要求转化为独立、可执行且兼顾合法行为的验证条件。本课题研究内容二围绕这一联系展开，重点研究根因推理、候选修复与安全性质验证之间的协同机制，以提升复杂真实漏洞修复的可靠性与可评价性。

2.3 基于智能体的漏洞修复验证

2.3.1 研究背景

漏洞修复是软件安全保障的重要环节，其目标不仅是消除已知漏洞的触发条件，还需要在维持合法功能的前提下阻断危险行为。围绕这一目标，自动漏洞修复（Automated Vulnerability Repair, AVR）研究形成了漏洞分析、补丁生成与补丁验证三个主要环节，采用程序分析、约束求解、程序变换和学习等技术降低对人工修复的依赖 [35]。随着大语言模型的发展，修复过程进一步由单一补丁生成扩展为交互式任务执行。例如，PatchAgent 将缺陷定位、补丁生成与验证组织到同一智能体中，利用语言服务器和验证工具支持多轮修复 [37]。这些研究主要回答“如何产生有效修复”，但软件系统的实际安全状态还取决于修复能否被正确集成并传播至最终部署的程序。

从软件供应链的角度看，漏洞修复远超上游提交补丁这一局部开发活动，它贯穿上游项目、下游分支、发行版、设备厂商与最终用户，构成一个持续维护过程。Li 和 Paxson 对安全补丁的研究表明，补丁开发需要同时考虑修复完整性与功能回归，安全修复也可能仅涉及少量代码变化 [67]。针对修复在相关仓库之间的传播，Machiry 等提出 SPIDER，通过分析补丁的行为影响，识别适合向相关项目传播的修复 [68]。Zhang 等对 Android 内核补丁生态的研究进一步发现，修复在多层供应链中的传播可能存在较长延迟 [69]。因此，“补丁已经公开”“下游已经合入”与“部署产物实际包含修复”是三个需要分别确认的状态。

版本信息和更新公告能够辅助判断修复状态，但不能在所有场景下替代程

序级证据。一方面，下游维护者可能将安全修复回移植至较早版本，同时保留原有上游版本号。Red Hat 的安全回移植实践明确指出，仅依据上游版本号开展漏洞扫描可能产生误判 [70]。另一方面，闭源产品、固件和第三方二进制组件往往无法提供与部署产物完全对应的源码，导致使用者难以直接检查安全修复是否进入最终程序。由此，针对实际二进制开展独立的修复状态核查，成为连接补丁发布与安全维护的重要技术需求 [71]。

补丁存在性测试（Patch Presence Testing, PPT）正是针对这一需求提出的：给定已知漏洞的修复信息，以及补丁状态未知的目标程序，判断目标中是否包含对应修复的实现证据。不同方法使用的参考材料有所区别，包括源码补丁、修复前后源码、参考二进制或中间表示等 [71, 72]。PPT 与通用二进制相似性检测的区别在于，后者主要寻找功能或结构相近的代码，前者则需要在高度相似的两个实现之间识别安全相关差异。一个边界条件、错误分支或参数处理的变化，可能不足以明显改变函数整体相似度，却足以改变其漏洞状态 [72]。

需要区分的是，**PPT 核查”特定修复是否体现在目标程序中”，而补丁正确性验证考察”修复是否充分消除目标缺陷并维持合法行为”**。即使检测到参考修复已经存在，也不能据此证明参考补丁本身完整，更不能推导出目标程序不存在其他漏洞 [35, 67, 71]。因此，本节将 PPT 定位为漏洞修复生命周期中的部署侧验证环节，重点讨论其二进制证据提取、补丁语义匹配以及智能体化分析能力。

2.3.2 研究现状与趋势

（一）基于局部语法与结构差异的补丁存在性测试 基于语法与结构差异的方法通过比较修复前后的程序变化构造补丁特征，再在目标中搜索相应特征。与直接比较整个函数相比，这类方法将分析范围收缩到补丁影响区域，以减少无关代码变化对判定的干扰 [72, 73]。

Xu 等提出的 BinXray 采用两阶段匹配：先通过函数签名检索疑似目标函数，再通过修复前后函数之间的基本块映射提取补丁签名，利用长度敏感的相似性比较区分漏洞版本与修复版本 [72]。其重要特点是分析可以在二进制层面完成，不要求提供目标源码，并将函数检索与补丁匹配纳入同一流程。由此可见，传统方法已经开始关注目标定位，但函数级相似性只能提供候选范围，仍需进一步识别区分修复前后状态的补丁级证据。

PatchDiscovery 进一步关注补丁签名的选择性。该方法在规范化和简化的控制流图上匹配基本块，分别提取漏洞版本与修复版本的关键基本块，再在目标函数中比较其匹配情况 [73]。与保留较多路径信息的签名相比，关键基本块能够减少无关指令和版本变化引入的噪声。这一路线的共同优势是特征直观、匹配流程相对明确，适合进行快速筛查；但局部结构特征仍需要在参考程序与目标程序之间保持可对应性。编译优化、基本块重组以及补丁之外的结构变化，都会使“同一修复”与“相似结构”之间的对应关系更加复杂 [72, 73]。

(二) 基于符号执行与中间表示的语义分析 为降低对指令形式的依赖，研究者开始利用符号执行和中间表示描述补丁引起的行为变化。Zhang 和 Qian 提出的 FIBER 结合源码变更分析与二进制符号执行，将选定的补丁位置转换为可匹配的二进制签名，并在匹配时结合局部控制流结构与语义公式 [71]。该方法通过结合结构约束与语义约束，增强签名对二进制表示差异的适应能力。FIBER 的实际评估使用带有符号信息的内核镜像；对于符号不可用的情形，论文讨论了接入相似性检索进行候选函数定位，但未在该工作中完成这一集成。

Jiang 等提出的 PDiff 面向经过厂商定制的下流内核，通过锚点基本块限定补丁影响路径，再利用路径约束、内存状态和函数调用等信息生成语义摘要，比较目标与修复前后参考版本的接近程度 [74]。该设计将比较对象从表面指令扩展为补丁相关路径上的行为，能够减轻定制代码的干扰。不过，其目标函数定位使用内核中的 KALLSYMS 信息，路径枚举也需要采取循环展开等近似策略，因此仍存在输入条件和分析范围上的边界。

Yang 等提出的 Robin 则通过符号执行寻找通向补丁区域的路径，构造能够区分修复前后行为的函数输入，并比较相同输入下的执行特征 [75]。它强调使用补丁引起的行为差异校验相似函数，而非仅依赖整体相似度。与此同时，函数级欠约束执行中得到的路径和输入，不必然对应完整程序中的真实可达执行，这也说明“行为特征相似”“补丁存在”与“漏洞可实际触发”需要分别讨论。

PS³ 通过符号模拟提取寄存器写入、内存存储、条件与函数调用等语义签名，利用参考与目标之间的签名比较降低编译选项变化的影响 [76]。REACT 将源码编译得到的 LLVM IR 与目标二进制提升得到的 IR 纳入统一分析流程，通过语义特征提取、特征细化与排序，以及约束求解进行补丁判定 [77]。这些方法拓展

了 PPT 的语义比较能力，但分析精度仍取决于符号模拟、二进制提升和特征抽象的准确性。IR 恢复误差与不完整的执行模型可能影响后续匹配，因此，即使采用约束求解，比较结论仍受所提取特征与建模条件的限制。

(三) 不同程序表示下的补丁检测与演化问题 除原生二进制外，Java 和 Android 字节码也形成了相应研究路线。BScout 直接检查 Java 可执行文件中的完整补丁，突破了仅依赖少量局部签名的传统做法 [78]；PPT4J 结合补丁的语义信息与字节码特征，建立源码补丁行和二进制实现之间的对应关系 [79]；PHunter 面向 Android 应用中的混淆代码，通过由粗到细的定位与匹配提升补丁识别能力 [80]。这些方法表明，分析对象保留的信息、代码变换方式以及定位粒度，会直接影响技术设计。字节码场景的能力不能不加区分地推广到经过优化和符号剥离的 C/C++ 原生二进制。

另一个需要独立考察的问题是安全补丁演化。Xie 等系统研究了补丁代码在后续软件版本中的持续变化，以及这些变化对已知漏洞分析工具的影响 [81]。编译差异主要改变同一源码实现的机器级表达，而补丁演化可能直接改变修复实现，例如重组检查逻辑或调整相关代码。因此，能够抵抗编译选项变化，不等于能够识别经过长期重构的修复。由此可见，传统方法的关键问题不仅是生成更加稳定的签名，还包括厘清签名所代表的安全行为，以及该行为在不同实现中成立所需的条件。

(四) 基于大语言模型的补丁语义分析 大语言模型为连接高层修复信息与低层程序表示提供了新的技术途径。DeGPT 利用模型改善反编译结果的可理解性，LLM4Decompile 探索从二进制代码生成或改进类源码表示，ReSym 则结合模型与程序分析恢复剥离二进制中的变量和数据结构符号 [82–84]。这些工作提供的是程序理解的辅助能力，并不直接回答特定安全补丁是否存在；它们的价值在于为跨表示的语义定位与比较提供了基础。

Li 等提出的 Lares 是直接面向 PPT 的 LLM 方法。它从修复前后源码提取补丁相关切片，补充数据依赖、控制依赖和宏信息，再由 LLM 将切片映射到目标函数的反编译伪代码；随后利用 Z3 检查可提取公式的等价性，并在不能直接判定时使用 LLM 比较语义 [85]。这种设计消除了为提取参考特征而重新编译历史版本的要求，将模型用于跨表示定位，而非仅作为最终二分类器。

Lares 的适用边界同样重要：其方法明确假设潜在漏洞函数已由外部二进制相似性方法识别，所解决的定位主要限于目标函数内部的代码切片定位，尚未覆盖从完整二进制独立完成全部检索步骤的场景。其失败分析还指出，重复检查与细微修改可能干扰判断 [85]。因此，“能够处理剥离后二进制的伪代码”与“不依赖任何定位提示完成端到端分析”并非同一能力；局部公式匹配也不能替代完整安全语义验证。

Xu 等提出的 PPTSP 采用另一种混合设计：先对语义等价的指令序列进行规范化，再提取补丁影响的关键控制路径以减少无关代码干扰，最后在结构特征相近时引入 LLM 判断语义等价性 [86]。与 Lares 将模型用于源码—伪代码映射不同，PPTSP 主要将模型用于传统特征处理之后的语义判别。两者体现出互补的结合方式：模型可以辅助定位候选证据，也可以处理已有特征难以区分的情况，但不同使用位置带来的收益与风险需要分别评估。

(五) 从 LLM 增强流水线到交互式智能体分析 将 LLM 加入某个分析阶段，与由智能体组织完整分析过程仍存在区别。SWE-agent 的研究表明，专门设计的智能体—计算机接口能够支持模型自主浏览仓库、执行程序并根据反馈调整操作 [50]。结合这一交互范式，可以进一步探索由智能体围绕补丁假设主动选择二进制分析工具，在候选检索、局部检查与证据补充之间迭代，避免始终沿固定顺序运行预定义模块。不过，通用代码任务与 PPT 在分析对象和证据要求上存在差异，这种交互能力能否转化为可靠的端到端补丁检测，仍需要专门研究。

这一变化还意味着评价对象需要从单次模型输出扩展为完整分析系统。模型、工具接口、上下文组织、停止条件与重试机制都会影响执行过程；有关智能体评价的研究也指出，仅关注任务准确率而忽视资源使用和复现条件，可能对系统实用性产生误判 [50, 87]。对 PPT 而言，需要进一步回答：模型能否自行找到正确代码，工具返回的信息能否构成补丁证据，失败发生在哪个环节，以及正确结论需要付出多少分析成本。因而，现有研究呈现出的发展方向已超越简单地“以 LLM 替代传统程序分析”，更强调将程序分析提供的可检查证据与模型的语义推理、任务规划能力结合，并对整条流程进行验证。

围绕上述问题，本人前期已开展两项相互衔接的实证研究。其一，针对现有 PPT 研究评估数据集有限且不一致、失败根因缺乏系统理解的问题，构建了包含

561 个 CVE、覆盖 10 个广泛使用开源项目的高保真基准，对五款代表性工具进行系统评估，揭示了“高准确率、低决策成功率”的普遍现象，并将失败模式归纳为算法层局限与工程层缺陷两大类 [88]。其二，针对智能体能否胜任端到端 PPT 这一问题，系统评估了通用编码智能体在符号可见、剥离二进制、演化补丁与真实发行版包等场景下的能力边界，发现前沿智能体仅使用 binutils 即可达到与专用 SOTA 系统相当的性能，但补丁定位与演化补丁识别仍是主要瓶颈 [7]。这两项工作作为研究内容三的提出提供了直接的实证基础。

2.3.3 研究挑战

(一) 缺少定位提示时，补丁语义与目标代码之间的可靠对应 现有方法在定位方面具有不同前提：FIBER 和 PDiff 的评估或实现利用目标符号信息，BinXray 包含函数级检索，Lares 则假设目标函数已由外部方法识别 [71, 72, 74, 85]。这些差异意味着，已知目标函数上的匹配效果不能直接代表完整二进制上的端到端效果。面向实际部署，分析不仅需要找到语义相关的函数，还必须确定具体检查作用于哪个对象、处于哪个程序位置，以及是否约束了对应危险操作。后续研究需要将函数检索、程序点定位与语义验证联系起来，并在候选存在歧义时继续收集证据，避免直接接受首个相似结果。

(二) 参考实现变化时，修复语义的保持与可观察性 语法签名、符号执行摘要和 IR 特征分别缓解了不同形式的程序差异，但它们仍基于一定的实现对应关系 [72–77]。补丁演化研究则表明，原始修复代码会在后续版本中持续变化 [81]。因此，需要进一步区分“原有补丁模式不再出现”与“修复效果不再存在”：前者可能只是实现调整，后者才涉及修复缺失。针对这一问题，应围绕检查对象、必要前提、错误传播和危险操作之间的关系组织验证，并按需扩展至调用上下文，避免仅停留在提高局部签名的匹配容忍度。同时，当给定分析范围内缺乏可区分证据时，应保留判断边界，不能以模型推测弥补不可观察信息。

(三) 模型推理与程序证据之间的一致性 Lares 和 PPTSP 将模型用于定位或语义判别，说明自然语言推理能够参与传统分析难以直接处理的环节 [85, 86]。但模型生成的解释、反编译输出以及求解器结论具有不同的证据性质：解释需对应实际代码，伪代码依赖反编译恢复质量，求解结论则受公式建模范围约

束 [77, 82, 84]。后续研究需要明确每个结论依赖哪些地址、语句、控制关系和数据关系，并针对竞争性解释寻找反证。还应区分未发现修复、漏洞相关功能不适用和分析未完成，避免将工具失败或检索无结果直接转换为漏洞状态判断。

(四) 不同研究条件下，能力与失败机制的可比较性 代表性 PPT 研究分别采用内核、通用 C/C++ 项目、Java 字节码或 Android 应用，并在参考材料、定位支持与编译条件上具有差异 [71–80]。因此，论文中的单一准确率或 F1 值不能脱离实验设置进行直接排序。需要在统一条件下分别考察特征生成、目标定位、语义判定与完整流程，明确是否提供符号、候选函数和调试信息，并同时记录错误判定、无法判定、超时与工具异常。只有保留这些失败情形，才能进一步判断局限源于补丁语义不可区分、实现假设不满足，还是工程处理过程不完整，避免将所有失败混为模型或算法精度问题。

(五) 智能体分析效果、工具配置与成本之间的权衡 交互式分析扩大了系统可以探索的证据空间，也增加了工具调用、上下文维护和重复推理的成本。通用智能体研究已经揭示接口设计和资源评价的重要性 [50, 87]，但不能由此预设 PPT 中“更大模型”“更多工具”或“更长上下文”必然带来更好效果。需要针对漏洞元数据、模型配置、工具能力与任务预算开展受控比较，研究如何优先使用低成本线索缩小范围，再针对关键证据缺口调用复杂分析，并同时评价正确性、完成率、时延和资源消耗。

综上，现有研究已经从补丁局部差异匹配发展到符号语义分析，并开始引入 LLM 进行跨表示定位与语义判别。然而，面向实际部署产物的可靠 PPT，仍需要贯通补丁理解、无充分提示的目标定位、演化修复识别和证据约束下的结论生成。对这些环节进行统一评估与失败机理分析，并探索程序分析和智能体推理之间的有效分工，是本课题研究内容三的主要切入点。

三 课题主要研究内容与预期目标

3.1 研究思路

漏洞挖掘、漏洞自动修复与漏洞修复验证构成了漏洞攻防闭环的三个关键环节，三者共同依赖于对漏洞语义、补丁语义与修复语义的深层理解。然而，现

有研究大多在”预定义规则 + 人工编排”的静态范式下分别推进三个环节，导致语义建模重复、能力难以复用、判定过度依赖专家经验。随着大语言模型与编码智能体的快速发展，一种以”模型推理 + 工具调用 + 迭代反馈”为核心的新型分析范式正在形成，为统一三个环节提供了契机。但与此同时，三个环节在智能体范式下仍各自面临深层语义推理瓶颈：在漏洞挖掘上，自因变量与语义 oracle 缺失制约着语义/逻辑漏洞的自动化发现；在漏洞修复上，部分规约下的修复正确性判定导致”补丁过拟合 PoC”问题普遍存在；在漏洞修复验证上，补丁定位与演化补丁识别仍是主要失分点。本文旨在提出基于智能体的漏洞挖掘与修复技术，通过统一建模漏洞—补丁—修复语义，并构建三个环节之间的能力复用与反馈闭环，实现高效、可靠的漏洞攻防自动化。

本课题的总体研究框架如图 1 所示，自下而上分为数据与基准层、智能体基础设施层、方法层与应用层四个层次。三个研究内容在闭环中形成有机整体：研究内容一（语义漏洞挖掘）产出的漏洞根因与 PoC 为研究内容二（漏洞自动修复）提供语义先验；研究内容二生成的候选补丁交由研究内容三（漏洞修复验证）进行修复正确性判定；研究内容三在验证过程中积累的安全性质规约又反过来增强研究内容一的语义 oracle 构造能力。

应用层：漏洞攻防闭环

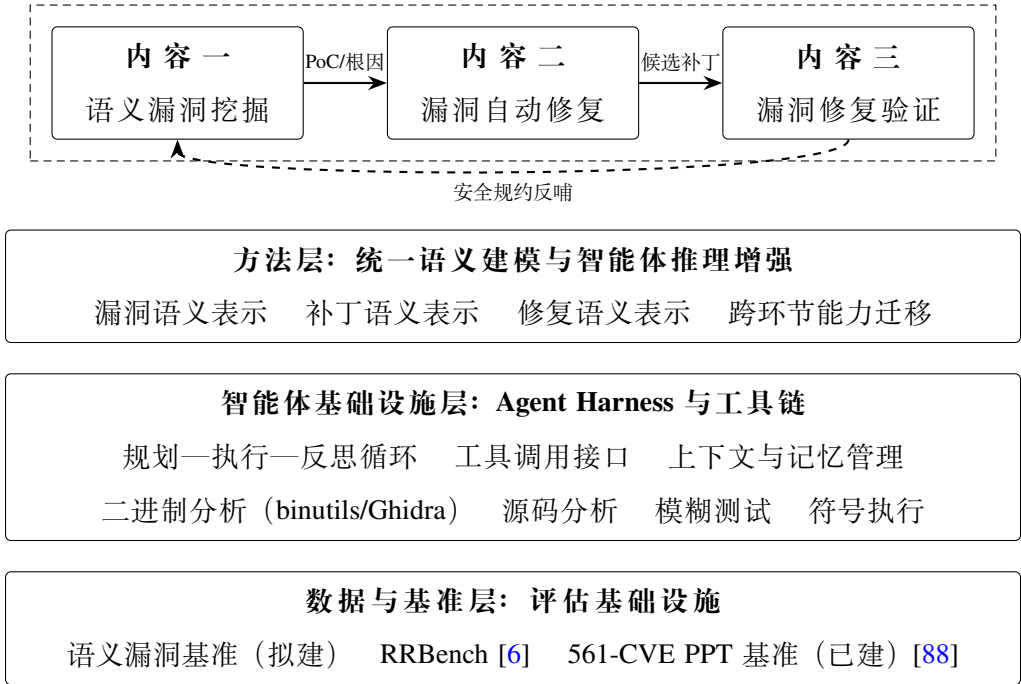


图 1 总体研究框架

针对基于智能体的语义漏洞挖掘，当前工作主要通过覆盖率引导的模糊测试与静态污点分析发现内存破坏类漏洞，存在对无崩溃信号的语义/逻辑漏洞检测能力不足的问题，因此面临**自因变量难以统一建模、语义 oracle 难以自动构造**的挑战。自因变量——那些不受外部输入直接控制、由内部状态、配置、时序与外设决定的变量——使传统输入驱动方法在错误的维度上搜索，而语义漏洞的判定又依赖人工构造的策略与规约。尽管已有工作在固件外设建模、协议状态感知与智能体辅助分析上取得进展 [13, 14, 22, 89]，但尚缺乏将自因变量显式对象化、并自动从自然语言文档提取安全不变式的系统化方法。

针对基于智能体的漏洞自动修复，当前工作主要通过“缺陷定位—补丁生成—测试验证”的单向流程合成补丁，存在对补丁是否真正消除漏洞根因缺乏可靠判定的问题，因此面临**部分规约下修复正确性难以验证、过拟合与回归难以系统抑制**的挑战。以 RRBench 为代表的大规模评估表明，现有智能体生成的“公开合理”补丁中有相当比例在私有安全性质检查下失败，且存在多个案例无法被任何系统正确修复 [6]。其根本原因在于修复环节缺乏对“修复意图”的语义判定能力，而这一能力恰与补丁存在性测试中的“补丁语义判定”高度同源，但现有研究尚未系统探索两者的对偶迁移。

针对基于智能体的漏洞修复验证，当前工作主要通过预定义语法特征或语义签名判断目标程序是否包含特定修复，存在对符号表与人工目标函数定位的强依赖、对演化补丁识别能力不足的问题，因此面临**跨抽象层次补丁语义对齐困难、修复意图难以形式化**的挑战。虽然前沿编码智能体已在端到端补丁存在性测试（Patch Presence Testing, PPT）上展现出与专用系统相当的性能 [7]，但其失败高度集中于剥离二进制中的补丁定位与演化补丁识别两个环节，且单次推理成本仍较高，难以同时支撑大规模软件供应链审计与修复正确性判定两类需求。

针对上述三个方面的挑战，本文计划如下开展研究工作：

针对**自因变量难以统一建模、语义 oracle 难以自动构造**的挑战，本文提出**基于智能体的语义漏洞挖掘方法**，通过构建统一的自因变量格以整合状态、配置、时序与外设等不同形态的内部变量，并利用智能体从自然语言文档自动提取安全不变式作为运行时 oracle，进而设计规划—执行—验证三智能体协同的模糊测试框架，实现语义/逻辑漏洞的高效自动化挖掘。

针对**部分规约下修复正确性难以验证、过拟合与回归难以系统抑制**的挑战，

本文提出**基于智能体的漏洞自动修复方法**，通过依托统一的漏洞语义五元组表示将漏洞报告与 PoC 转化为结构化根因语义，利用智能体自动提取补丁必须满足的安全性质规约并引导多轮补丁生成，合成高质量候选补丁，为修复验证环节提供规约与判定依据。

针对**跨抽象层次补丁语义对齐困难、修复意图难以形式化**的挑战，本文提出**基于智能体的漏洞修复验证方法**，通过离线提取”触发条件—危险操作—干预点—修复意图—上下文”的漏洞语义五元组，在线引导智能体进行粗细两阶段补丁定位与基于修复意图的语义等价判定，实现剥离二进制与演化补丁场景下高决策成功率、低推理成本的端到端修复状态验证，并将补丁语义判定能力对偶迁移至研究内容二的修复正确性验证。

3.2 研究内容一：基于智能体的语义漏洞挖掘方法

主要研究内容：语义/逻辑漏洞（如认证绕过、状态机错误、配置驱动缺陷）不产生崩溃信号，长期缺乏高效的自动化挖掘手段，是 IoT 与嵌入式固件安全中的突出短板。其核心难点在于：此类漏洞的触发路径通常由内部状态、配置、时序与外设响应等**自因变量**共同决定，传统输入驱动的模糊测试难以在错误的维度上有效探索；同时，语义漏洞的判定依赖人工构造的策略与规约，oracle 的缺失使自动化检测难以规模化。本研究内容旨在通过统一建模自因变量、自动构造语义 oracle、设计自因变量感知的智能体模糊测试，突破语义漏洞挖掘的自动化瓶颈，扩展可检测漏洞的类型与规模。

当前工作存在**自因变量难以统一建模、语义 oracle 难以自动构造**的挑战，因此，本文提出**自因变量感知的多智能体语义漏洞挖掘框架**。具体而言，首先综合静态分析与智能体推理，在固件二进制上识别各类自因变量，并统一表示为标注来源类型、取值空间与影响路径的自因变量格，使其成为模糊测试的一等探索维度；其次，利用智能体从协议 RFC、芯片数据手册与配置说明等自然语言文档中自动提取安全不变式，并编译为可执行的运行时监测点与断言，解决 oracle 自动构造难题；最后，设计规划—执行—验证三智能体协同的模糊测试架构，由规划智能体根据覆盖率与不变式反馈规划探索策略，执行智能体在生成输入的同时主动翻转目标自因变量（如注入中断、修改 NVRAM、模拟外设响应），验证智能体检查不变式违背并报告候选漏洞，并配套 PoC 生成与根因定位能力。

预期目标：构建基于智能体的语义漏洞挖掘系统，在真实 IoT 固件与合成基准上验证有效性，相比现有最优模糊测试方法在语义漏洞发现数量上提升 ≥ 5 倍，自因变量覆盖率 $\geq 70\%$ ，安全 oracle 自动构造准确率 $\geq 80\%$ ，在认证绕过、状态机错误与配置驱动缺陷三类典型语义漏洞上各发现若干真实漏洞并获得 CVE 编号，形成可复用的固件语义漏洞基准与工具原型。

3.3 研究内容二：基于智能体的漏洞自动修复方法

主要研究内容：漏洞自动修复是连接漏洞发现与漏洞消除的关键环节，其目标是自动定位漏洞根因、合成补丁并验证正确性。以 RRBench 为代表的大规模评估揭示了一个关键问题：现有智能体生成的“公开合理”补丁中，有相当比例（23.4%）在私有安全性质检查下失败，且存在多个案例无法被任何系统正确修复 [6]。其根本原因在于：修复环节缺乏对“修复意图”的语义判定能力，智能体容易生成仅抑制特定 PoC、却未消除漏洞根因的过拟合补丁。值得注意的是，这一问题与补丁存在性测试中的“补丁语义判定”在本质上是对偶的——前者回答“生成的补丁是否构成有效修复”，后者回答“目标二进制是否包含修复”——两者均需对漏洞语义与修复语义进行深层建模。本研究内容旨在构建规约引导的可信自动修复框架，生成高质量候选补丁，并与研究内容三的修复验证方法形成“生成—验证”协同。

当前工作存在**部分规约下修复正确性难以验证、过拟合与回归难以系统抑制**的挑战，因此，本文提出**规约引导的智能体漏洞修复框架**。具体而言，首先依托统一的漏洞语义五元组表示（与研究内容一、三共享），将漏洞报告与 PoC 转化为结构化的根因语义表示；其次，借鉴 RRBench 私有 Oracle 的设计思想但将其自动化，利用智能体基于漏洞语义与代码上下文自动提取补丁必须满足的安全性质集合，既指导补丁生成又作为最终验证依据；再次，修复智能体在根因语义与安全规约的双重引导下进行多轮补丁生成与迭代，通过编译、公开测试与静态检查反馈持续修正候选补丁；最后，候选补丁交由研究内容三的修复验证环节进行 PPT 式的修复正确性判定，二者构成“生成—验证”对偶闭环，系统性抑制过拟合补丁与合法功能回归，实现部分规约下可信的自动修复。

预期目标：构建基于智能体的漏洞自动修复系统，在 RRBench 基准上将正确修复率从当前最优 69.6% 提升至 $\geq 80\%$ ，私有验证拒绝率降低 $\geq 50\%$ ，在现有

任何系统均无法正确修复的 14 个案例上成功修复 ≥ 4 个，形成可复用的安全规约自动提取工具链，并将方法论扩展至 C/C++ 等其他语言生态的修复基准。

3.4 研究内容三：基于智能体的漏洞修复验证方法

主要研究内容：补丁存在性测试（Patch Presence Testing, PPT）是连接漏洞披露与实际防护的关键环节，在软件供应链安全审计中具有不可替代的作用；同时，修复验证也是漏洞自动修复闭环的收官环节——自动合成的补丁是否真正消除漏洞根因，最终需要同样的补丁语义判定能力来回答。本人前期实证研究表明，前沿编码智能体已在端到端补丁存在性测试上达到与专用 SOTA 系统相当的性能 [7]，但其失败高度集中于两个环节：剥离二进制中的补丁定位，以及对后续版本中演化补丁的识别。前者源于源码级补丁意图与二进制级指令序列之间存在跨抽象层次的语义鸿沟，后者源于现有方法过度依赖补丁的形式特征而非修复意图。本研究内容旨在通过引入统一的漏洞语义表示，增强智能体在两个瓶颈环节上的推理能力，推动端到端修复状态验证从“可用”走向“可信、可扩展”，并将补丁语义判定能力对偶迁移至自动修复的补丁正确性验证。

当前工作存在**跨抽象层次补丁语义对齐困难、修复意图难以形式化**的挑战，因此，本文提出**语义先验引导的智能体修复验证增强框架**。具体而言，离线阶段利用智能体从 CVE 描述、补丁 diff 与源码上下文中提取“触发条件—危险操作—干预点—修复意图—上下文”的漏洞语义五元组，作为统一语义先验；在线阶段采用“粗—细”两阶段补丁定位，先用启发式规则快速缩小候选函数集，再以五元组为约束引导智能体在反汇编切片中进行置信度排序；针对演化补丁，提出基于修复意图的语义等价判定方法，不再追求形式上的补丁匹配，而是验证目标二进制中的干预是否实现了与原始补丁相同的安全语义。该框架在保持推理成本可控的前提下，系统性提升补丁定位准确率与演化补丁识别能力。

预期目标：构建基于智能体的端到端漏洞修复验证系统，在 561-CVE 基准上实现 $DSR \geq 0.98$ ，在演化补丁子集上 DSR 提升 $\geq 15\%$ ，在 Ubuntu 真实发行版包上保持 $DSR \geq 0.95$ ，单例推理成本相比前沿配置增加不超过 10%，形成可支撑大规模软件供应链审计与修复正确性判定的工具原型。

3.5 预期目标与创新点

(待撰写：分点列出理论创新、方法创新与系统/工具成果，以及预期论文与专利产出。)

四 拟采用的研究方法、技术路线、实验方案及其可行性分析

4.1 基于智能体的语义漏洞挖掘方法

拟解决的关键问题：语义/逻辑漏洞（认证绕过、状态机错误、配置驱动缺陷）自动化挖掘的两大瓶颈——**自因变量难以统一建模与语义 oracle 难以自动构造**。自因变量——那些不受外部输入直接控制、由内部状态、配置、时序与外设决定的变量——在文献中以 state-driven、config-driven、timing-driven、peripheral-driven 等不同形态各自为战，缺乏统一抽象；同时，语义漏洞的判定依赖人工构造的策略与规约，Firmalice 的认证绕过 oracle 使用十年仍是孤例 [12]，oracle 的缺失使自动化检测难以规模化。

技术方案：针对上述问题，本文提出**自因变量感知的多智能体语义漏洞挖掘框架**，整体技术路线如图 2 所示，包含四个核心模块：

- **模块一：自因变量统一建模。**综合静态分析与智能体推理，在固件二进制上识别各类自因变量，并统一表示为标注来源类型、取值空间与影响路径的自因变量格，使后续 fuzzing 能够将自因变量作为一等探索维度。
- **模块二：语义 oracle 自动构造。**利用智能体从自然语言文档（协议 RFC、芯片数据手册、配置说明）中自动提取安全不变式，并将其编译为可执行的运行时监测点与断言，解决语义漏洞挖掘中“oracle 靠人工”的长期瓶颈。
- **模块三：自因变量感知的智能体 fuzzing。**采用规划—执行—验证三智能体协同架构：规划智能体根据覆盖率与不变式反馈规划探索策略；执行智能体在生成输入的同时，主动翻转目标自因变量；验证智能体检查不变式违背。该模块突破传统 fuzzing 仅在输入空间探索的局限。
- **模块四：漏洞确认与根因定位。**对验证智能体报告的候选漏洞，自动生成 PoC，通过动态切片定位根因，并提供人工审核接口。



图 2 基于智能体的语义漏洞挖掘技术路线

可行性分析：(1) 理论可行：自因变量概念已在 SyzTrust [90]、Fuzzware [13] 等工作中以不同形态出现，本方案是对这些分散实践的统一与提升；(2) 技术可行：多智能体协同架构已在 FirmAgent [22] 中得到初步验证，LLM 的规约理解能力已在 ChatAFL [19] 等工作中展现，”语义检索 + 可验证推理”的 agentic 范式已在通用二进制分析任务上被证明有效 [21]；(3) 工程可行：课题组在固件分析、智能体框架与 fuzzing 工具链上有完整积累，RRBench 的”角色分离智能体”设计模式可直接复用 [6]；(4) 数据可行：可通过公开固件数据集与 FreeRTOS、Zephyr 等开源 IoT 项目获取评估对象，并可人工植入语义漏洞构建合成基准。

实验方案：基准包括 10–15 个含已知语义漏洞的真实 IoT 固件（含认证绕过、状态机错误等）与参考 LAVA-M 设计的 50+ 合成植入漏洞；对比方法包括 FIRM-AFL [91]、Fuzzware [13]、FirmAgent [22]、ChatAFL [19]、AFLNet [14]；评估指标包括语义漏洞发现数量、首次发现时间、假阳性率、自因变量覆盖率、oracle 自动构造准确率；消融实验分别验证自因变量建模、自动 oracle 与多智能体协同的独立贡献。预期语义漏洞发现数量相比最优基线提升 ≥ 5 倍，自因变量覆盖率 $\geq 70\%$ ，oracle 自动构造准确率 $\geq 80\%$ 。

4.2 基于智能体的漏洞自动修复方法

拟解决的关键问题：智能体漏洞自动修复的两大瓶颈——**部分规约下修复正确性难以验证与过拟合与回归难以系统抑制**。RRBench 大规模评估表明，现有智能体生成的”公开合理”补丁中 23.4% 在私有安全性质检查下失败，且 14 个案例未被任何系统正确修复 [6]。其根本原因在于：修复环节缺乏对”修复意图”的语义判定能力，智能体容易生成仅抑制特定 PoC、却未消除漏洞根因的过拟合补丁，或在修复漏洞的同时破坏合法功能。

技术方案：针对上述问题，本文提出**修复—验证对偶的智能体漏洞修复框架**，核心思想是将 PPT 的”补丁语义判定”能力对偶迁移为”修复正确性验证”，整体技术路线如图 3 所示，包含四个核心模块：

- **模块一：漏洞根因语义理解。**依托统一的漏洞语义五元组表示（与研究内容三共享），将漏洞报告与 PoC 转化为结构化的根因语义表示，为后续修复提供深层理解基础。
- **模块二：安全性质规约自动提取。**这是连接 PPT 与修复的关键创新。借鉴



图 3 基于智能体的漏洞自动修复技术路线

RRBench 私有 Oracle 的设计思想，但将其从”人工构造”扩展为”智能体自动生成”：智能体基于漏洞语义与代码上下文，自动提取补丁必须满足的安全性质集合。这些规约既可指导补丁生成，也可作为最终验证依据。

- **模块三：规约引导的补丁生成。**修复智能体在根因语义与安全规约的双重引导下生成候选补丁，并通过编译、公开测试与静态检查进行多轮迭代。
- **模块四：PPT 式修复验证。**这是本研究内容的核心创新，将 PPT 能力对偶迁移到修复验证：通过**补丁语义判定**（验证候选补丁是否真正实现应有不变式）、**差分测试**（自动构造与 PoC 语义相近但不同的输入以检测过拟合）、**回归检查**（验证合法功能保持）三重机制，系统性抑制过拟合补丁与合法功能回归。

可行性分析：（1）直接基于 RRBench：基准、评估框架、五款基线 agent 数据均已就绪，可快速开展对比实验，与课题组前期工作延续性强 [6]；（2）PPT 能力可迁移：研究内容三的”修复意图判定”与本内容的补丁验证需求本质同源，技术路线已形成闭环，风险分散；（3）规约自动提取有先例：ChatAFL 的协议规

约提取、Firmallice 的特权模型等已证明 LLM 具备规约理解能力 [12, 19]; (4) 工程基础成熟: SWE-agent [50]、OpenHands [92] 等开源框架提供了成熟的修复智能体底座。

实验方案: 基准采用 RRBench 全部 115 个案例, 后续扩展至 SEC-bench [55]、CVE-Bench [5] 子集; 对比方法包括 SWE-agent [50]、OpenHands [92]、RepairAgent [51]、RustAgent、RustForger (即 RRBench 原五款); 评估指标包括 Correct Repair Rate (通过公开 + 私有验证)、Plausible-but-Private-Rejected Rate、平均迭代轮数、每例成本; 消融实验分别验证安全规约引导、PPT 式验证与差分测试的独立贡献。预期 Correct Repair Rate $\geq 80\%$, 私有拒绝率降低 $\geq 50\%$, 在 14 个“无人能修”案例上成功修复 ≥ 4 个。

4.3 基于智能体的漏洞修复验证方法

拟解决的关键问题: 智能体在端到端漏洞修复验证 (以补丁存在性测试为核心任务) 中的两大瓶颈——**剥离二进制中的补丁定位与演化补丁识别**。前期实证研究表明, 前者占弱模型配置独有失败的 95.2%, 后者是鲁棒性测试的主要失分点 [7]。两者本质上都源于跨抽象层次补丁语义对齐困难、修复意图难以形式化: 源码级补丁意图与二进制级指令序列之间存在语义鸿沟, 且现有方法过度依赖补丁的形式特征而非修复意图。

技术方案: 针对上述问题, 本文提出**语义先验引导的智能体修复验证增强框架**, 整体技术路线如图 4 所示, 包含三个核心模块:

- **模块一: 漏洞语义先验提取。** 离线阶段利用智能体从 CVE 描述、补丁 diff 与源码上下文中提取漏洞语义五元组, 作为统一语义先验, 替代现有方法仅依赖原始补丁 diff 的浅层表示。
- **模块二: 语义引导的补丁定位。** 在线阶段采用“粗—细”两阶段定位: 先用传统启发式 (字符串、常量、导入表) 快速缩小候选函数集, 再将五元组作为约束注入智能体推理, 引导其在候选函数中进行语义切片与置信度排序, 输出 Top-K 补丁候选区域。
- **模块三: 演化补丁的语义等价判定。** 针对同一漏洞可能存在多种实现形式的修复 (重构、拆分、合并等), 不再追求“形式上的补丁匹配”, 而是让智



图 4 基于智能体的漏洞修复验证技术路线

智能体判断目标二进制中的干预点是否实现了与原始补丁相同的修复意图，并通过语义不变式进行交叉验证。

可行性分析：（1）数据基础充分：已构建 561 CVE 高保真基准与 12 组智能体配置的运行日志 [7, 88]，包含大量补丁定位与演化补丁的成功/失败案例，可直接用于方法设计与消融验证；（2）技术路线有实证支撑：前期工作已证明“丰富补丁元数据提升 DSR”与“补丁定位是主要瓶颈”两个关键结论，本方案是这两个结论的直接延伸，技术风险低；（3）工具链成熟：智能体框架（Codex / MiniSWEAgent）、二进制分析工具（binutils / Ghidra）与评估基准均已就绪，可快速进入实验迭代。

实验方案：基准沿用前期 561-CVE Source-Built Benchmark + Patch Evolution Set + Ubuntu Packages Set；对比方法包括 Codex 原始配置、Lares [85]、PPTSP [86]、PS³ [76]、BinXray [72]；评估指标包括 DSR、Accuracy、补丁定位准确率、每例 token 成本与时延；消融实验分别验证三个模块的独立贡献。预期全配置 DSR ≥ 0.98，演化补丁子集 DSR 提升 ≥ 15%，成本增加 ≤ 10%；进一步将验证能力对偶迁移至修复场景，在 RRBench 生成补丁上验证修复正确性判定的有效性。

4.4 实验方案设计

4.4.1 整体评估策略

三个研究内容的实验遵循统一策略：

1. **基准优先**：每个内容均基于已有或新建的高质量基准，确保可复现与可比较。
2. **多维度评估**：从效果（准确率/发现数/修复率）、效率（时间/token 成本）、鲁棒性（跨配置/跨场景）三个维度综合评估。
3. **消融驱动**：每个核心模块均设计消融实验，验证其独立贡献。
4. **跨内容联动**：设计端到端实验，验证三个内容构成的闭环是否产生协同增益。

4.4.2 端到端闭环验证实验

为验证三个研究内容构成的攻防闭环的有效性，设计如下端到端实验：

1. **输入**：一批含已知漏洞的 IoT 固件镜像（部分为真实 CVE，部分为人工植入）。
2. **流程**：内容一挖掘未知语义漏洞，产出根因与 PoC；内容二对确认的漏洞进行自动修复，产出候选补丁；内容三对修复后固件进行 PPT 式修复验证，判定补丁是否真正消除漏洞根因；同时内容三对固件中已知 CVE 的修复状态进行补丁存在性核查，覆盖供应链审计场景。
3. **评估**：闭环整体准确率、各环节能力复用带来的效率提升、相比独立运行的性能差异。

4.4.3 基准建设计划

4.4.4 风险与应对

表 1 基准建设计划

基准	状态	规模	用途
561-CVE PPT 基准	已建成	561 CVE / 10 项目	内容三
语义漏洞固件基准	拟建设	10–15 真实固件 + 50 合成案例	内容一
RRBench	已建成 (课题组)	115 案例 / 95 仓库	内容二
端到端闭环基准	拟整合	复用上述 + 新增修复验证	闭环验证

表 2 风险与应对策略

风险	影响	应对策略
语义 oracle 自动构造准确率不足	内容一假阳性率高	引入人工审核接口；采用多智能体交叉验证
自因变量建模在复杂固件上扩展性差	内容一覆盖率不达预期	从中小型固件起步逐步扩展；结合动态污点辅助识别
修复验证的私有规约生成不完整	内容二私有拒绝率下降有限	结合官方补丁对比学习；引入形式化方法辅助规约补全
智能体推理成本过高	方法难以规模化	分级模型策略（轻量粗筛 + 重量精判）；缓存中间语义表示

五 已有科研基础与所需的科研条件

5.1 已有科研基础

本人已经针对本文涉及的相关研究进行了充分调研和分析，积累了相应理论和实践基础。

针对研究内容一（基于智能体的语义漏洞挖掘），本人正在开展相关调研与原型设计工作。

针对研究内容二（基于智能体的漏洞自动修复），本人深度参与课题组 RRBench 基准的讨论与分析工作，熟悉其两阶段评估设计与五款基线智能体的运行数据 [6]，为后续修复研究奠定了直接基础。

针对研究内容三（基于智能体的漏洞修复验证），本人已完成两项相互衔接的实证研究：（1）构建了包含 561 个 CVE、覆盖 10 个广泛使用开源项目的高保真 PPT 基准，对五款代表性工具进行了系统评估，揭示了“高准确率、低决策成功率”的普遍现象，并提出改进策略取得显著提升，成果发表于 ISSTA 2026 [88]；（2）系统评估了编码智能体在端到端 PPT 任务中的能力边界与配置空间，识别出补丁定位与演化补丁识别两大瓶颈，成果已完成论文撰写 [7]。

（待补充：项目参与情况、其他安全分析实践成果。）

5.2 所需科研条件

（待撰写：计算资源（GPU 集群与 LLM API 预算）、数据资源（固件语料、CVE 数据库）、团队与项目支撑。）

六 研究工作计划与进度安排

1. **2026.09 – 2027.03**：完成研究内容一。构建自因变量统一建模与语义 oracle 自动构造方法，实现多智能体语义漏洞挖掘原型，建设固件语义漏洞基准，撰写并发表研究内容一论文。
2. **2027.03 – 2027.12**：完成研究内容二。实现规约引导的智能体漏洞修复框架，在 RRBench 上完成系统评估，撰写并发表研究内容二论文。
3. **2028.01 – 2028.10**：完成研究内容三。设计并实现语义先验引导的智能体修

复验证（PPT 增强）框架，在 561-CVE 基准上完成对比与消融实验，撰写并发表研究内容三论文。

4. **2028.10 – 2029.06**：完成三个研究内容的端到端闭环集成与验证，整理成果，撰写博士学位论文，准备答辩。

（注：以上时间节点为初步规划，可按实际学制与导师意见调整。）

参考文献

- [1] Apache Software Foundation. Apache Log4j CVE-2021-44228 [EB/OL]. 2021. <https://nvd.nist.gov/vuln/detail/CVE-2021-44228>.
- [2] Google Project Zero and DeepMind. From naptime to big sleep: Using large language models to catch vulnerabilities in real-world code [EB/OL]. 2024. <https://googleprojectzero.blogspot.com/2024/10/from-naptime-to-big-sleep.html>.
- [3] DARPA. AI cyber challenge (AIXCC) [EB/OL]. 2025. <https://aicyberchallenge.com/>.
- [4] Jimenez C E, Yang J, Wettig A, et al. SWE-bench: Can language models resolve real-world GitHub issues? [C]//International Conference on Learning Representations (ICLR). 2024.
- [5] Anonymous. CVE-Bench: A benchmark for AI agents' ability to exploit real-world web application vulnerabilities [Z]. 2024.
- [6] Anonymous. RRBench: A repository-level benchmark for agentic repair of real-world Rust vulnerabilities [Z]. 2026.
- [7] Zhang X, et al. Understanding coding agents for patch presence testing [Z]. 2026.
- [8] Manès V J M, Han H, Han C, et al. The art, science, and engineering of fuzzing: A survey [J/OL]. IEEE Transactions on Software Engineering, 2019, 47(11): 2312-2331. DOI: [10.1109/TSE.2019.2946563](https://doi.org/10.1109/TSE.2019.2946563).
- [9] Serebryany K, Bruening D, Potapenko A, et al. AddressSanitizer: A fast address sanity checker [C]//2012 USENIX Annual Technical Conference. 2012: 309-318.
- [10] Mozilla. Frontier-model-assisted vulnerability discovery in Firefox [Z]. 2026.
- [11] Barr E T, Harman M, McMinn P, et al. The oracle problem in software testing: A survey [J/OL]. IEEE Transactions on Software Engineering, 2015, 41(5): 507-525. DOI: [10.1109/TSE.2014.2370955](https://doi.org/10.1109/TSE.2014.2370955).
- [12] Shoshitaishvili Y, Wang R, Hauser C, et al. Fomalice: Automatic detection of authentication bypass vulnerabilities in binary firmware [C]//Network and Distributed System Security Symposium (NDSS). 2015.

- [13] Scharnowski T, Bars N, Schloegel M, et al. Fuzzware: Using precise MMIO modeling for effective firmware fuzzing [C]//31st USENIX Security Symposium. 2022: 1239-1256.
- [14] Pham V T, Böhme M, Roychoudhury A. AFLNet: A greybox fuzzer for network protocols [C]//2020 IEEE 13th International Conference on Software Testing, Validation and Verification (ICST). 2020: 460-465.
- [15] Natella R. StateAFL: Greybox fuzzing for stateful network servers [C]//2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN). 2022.
- [16] Meng R, Mirchev M, Böhme M, et al. Linear-time temporal logic guided greybox fuzzing [C/OL]//Proceedings of the 44th International Conference on Software Engineering. 2022: 765-777. DOI: [10.1145/3510003.3510082](https://doi.org/10.1145/3510003.3510082).
- [17] Yao J, Liu M, Chen Z, et al. Context-free property oriented fuzzing [C]//Proceedings of the 48th International Conference on Software Engineering. 2026.
- [18] Xia C S, Palma F, Jia M L, et al. Fuzz4All: Universal fuzzing with large language models [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024. DOI: [10.1145/3597503.3639121](https://doi.org/10.1145/3597503.3639121).
- [19] Meng R, Mirchev M, Böhme M, et al. Large language model guided protocol fuzzing [C]//Network and Distributed System Security Symposium (NDSS). 2024.
- [20] Androutsopoulos G, Bianchi A. deepSURF: Detecting memory safety vulnerabilities in Rust through fuzzing LLM-augmented harnesses [C]//2026 IEEE Symposium on Security and Privacy. 2026.
- [21] Liu Y, Liu Z, Yu Z, et al. Towards generality: Task-adaptive binary analysis via semantic retrieval and verifiable reasoning [C]//35th USENIX Security Symposium. 2026.
- [22] Ji J, Zhang C, Gan S, et al. FirmAgent: Leveraging fuzzing to assist LLM agents with IoT firmware vulnerability discovery [C/OL]//Network and Distributed System Security Symposium (NDSS). 2026. DOI: [10.14722/ndss.2026.231943](https://doi.org/10.14722/ndss.2026.231943).
- [23] Jia Z, et al. Fuzzing multilingual IoT firmware with LLM-driven code analysis [C]//35th USENIX Security Symposium. 2026.
- [24] Wang Y, Zhang W, Liang R, et al. FIRMENSTEIN: Scaling dynamic analysis for Linux-based firmware services via API-centric intervention code synthesis [C]//35th USENIX Security Symposium. 2026.
- [25] Peng J, Liu P, Cheng K, et al. Bond: Constraint-directed fuzzing for automated validation of taint analysis results in Linux-based IoT firmware [C]//35th USENIX Security Symposium. 2026.
- [26] Yang T, Qin Y, Zhang L, et al. BSFuzzer: Context-aware semantic fuzzing for BLE logic flaw detection [C]//Network and Distributed System Security Symposium (NDSS). 2026.

- [27] Song X, Pei L, Wu J, et al. ProtocolGuard: Detecting protocol non-compliance bugs via LLM-guided static analysis and dynamic verification [C]//Network and Distributed System Security Symposium (NDSS). 2026.
- [28] Potyondy T, Tarbinian A, Schuermann L, et al. Rage against the state machine: Type-stated hardware peripherals for increased driver correctness [C/OL]//Proceedings of the 31st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS). 2026. DOI: [10.1145/3779212.3790207](https://doi.org/10.1145/3779212.3790207).
- [29] Wang R, Li T, Li J. TyPatch: Transforming patches into typestate rules for kernel bug detection [Z]. 2026.
- [30] Yang R, He Y, Xu Y, et al. LLM-based invariant testing for software functional bugs [C]//2026 IEEE International Symposium on Software Reliability Engineering (ISSRE). 2026.
- [31] Zhang K, Xiao D, Wu D, et al. LLM-powered silent bug fuzzing in deep learning libraries via versatile and controlled bug transfer [J]. Proceedings of the ACM on Programming Languages, 2026, 10(OOPSLA1).
- [32] Wen C, Wang H, Li Y, et al. MemLock: Memory usage guided fuzzing [C/OL]//Proceedings of the 42nd International Conference on Software Engineering. 2020: 765-777. DOI: [10.1145/3377811.3380411](https://doi.org/10.1145/3377811.3380411).
- [33] Wang C, Liu J, Peng X, et al. Boosting static resource leak detection via LLM-based resource-oriented intention inference [C]//Proceedings of the IEEE/ACM 47th International Conference on Software Engineering. 2025.
- [34] Google. Leaksanitizer [EB/OL]. 2026. <https://github.com/google/sanitizers/wiki/AddressSanitizerLeakSanitizer>.
- [35] Hu Y, Li Z, Shu K, et al. Sok: Automated vulnerability repair: Methods, tools, and assessments [C]//34th USENIX Security Symposium. 2025: 4421-4440.
- [36] Smith E K, Barr E T, Le Goues C, et al. Is the cure worse than the disease? overfitting in automated program repair [C/OL]//Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering. 2015: 532-543. DOI: [10.1145/2786805.2786825](https://doi.org/10.1145/2786805.2786825).
- [37] Yu Z, Guo Z, Wu Y, et al. PatchAgent: A practical program repair agent mimicking human expertise [C]//34th USENIX Security Symposium. 2025: 4381-4400.
- [38] Jung R, Jourdan J H, Krebbers R, et al. RustBelt: Securing the foundations of the Rust programming language [J/OL]. Proceedings of the ACM on Programming Languages, 2018, 2(POPL): 1-34. DOI: [10.1145/3158154](https://doi.org/10.1145/3158154).
- [39] Qin B, Chen Y, Yu Z, et al. Understanding memory and thread safety practices and issues in real-world Rust programs [C/OL]//Proceedings of the 41st ACM SIGPLAN Conference on

- Programming Language Design and Implementation. 2020: 763-779. DOI: [10.1145/3385412.3386036](https://doi.org/10.1145/3385412.3386036).
- [40] Le Goues C, Nguyen T, Forrest S, et al. GenProg: A generic method for automatic software repair [J]. IEEE Transactions on Software Engineering, 2012, 38(1): 54-72.
 - [41] Nguyen H D T, Qi D, Roychoudhury A, et al. SemFix: Program repair via semantic analysis [C/OL]//2013 35th International Conference on Software Engineering. 2013: 772-781. DOI: [10.1109/ICSE.2013.6606623](https://doi.org/10.1109/ICSE.2013.6606623).
 - [42] Gao X, Wang B, Duck G J, et al. Beyond tests: Program vulnerability repair via crash constraint extraction [J/OL]. ACM Transactions on Software Engineering and Methodology, 2021, 30(2): 1-27. DOI: [10.1145/3418461](https://doi.org/10.1145/3418461).
 - [43] Zhang Y, Gao X, Duck G J, et al. Program vulnerability repair via inductive inference [C/OL]//Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis. 2022: 691-702. DOI: [10.1145/3533767.3534387](https://doi.org/10.1145/3533767.3534387).
 - [44] Fu M, Tantithamthavorn C, Le T, et al. VulRepair: A T5-based automated software vulnerability repair [C/OL]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 935-947. DOI: [10.1145/3540250.3549098](https://doi.org/10.1145/3540250.3549098).
 - [45] Zhou X, Kim K, Xu B, et al. Out of sight, out of mind: Better automatic vulnerability repair by broadening input ranges and sources [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-13. DOI: [10.1145/3597503.3639222](https://doi.org/10.1145/3597503.3639222).
 - [46] Pearce H, Tan B, Ahmad B, et al. Examining zero-shot vulnerability repair with large language models [C/OL]//2023 IEEE Symposium on Security and Privacy. 2023: 2339-2356. DOI: [10.1109/SP46215.2023.10179420](https://doi.org/10.1109/SP46215.2023.10179420).
 - [47] Wen X C, Lin Z, Yang Y, et al. Vul-R2: A reasoning LLM for automated vulnerability repair [C/OL]//2025 40th IEEE/ACM International Conference on Automated Software Engineering. 2025: 26-38. DOI: [10.1109/ASE63991.2025.00011](https://doi.org/10.1109/ASE63991.2025.00011).
 - [48] Nong Y, Yang H, Cheng L, et al. APPATCH: Automated adaptive prompting large language models for real-world software vulnerability patching [C]//34th USENIX Security Symposium. 2025: 4481-4500.
 - [49] Li Z, Wang N, Zou D, et al. On the effectiveness of function-level vulnerability detectors for inter-procedural vulnerabilities [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639218](https://doi.org/10.1145/3597503.3639218).
 - [50] Yang J, Jimenez C E, Wettig A, et al. SWE-agent: Agent-computer interfaces enable automated software engineering [C]//Advances in Neural Information Processing Systems: volume 37. 2024: 50528-50652.

- [51] Bouzenia I, Devanbu P, Pradel M. RepairAgent: An autonomous, LLM-based agent for program repair [C/OL]//2025 IEEE/ACM 47th International Conference on Software Engineering. 2025: 2188-2200. DOI: [10.1109/ICSE55347.2025.00157](https://doi.org/10.1109/ICSE55347.2025.00157).
- [52] Wang H, Basque Z L, Hu J, et al. Root-cause-driven automated vulnerability repair [EB/OL]. 2026. DOI: [10.48550/arXiv.2605.04251](https://doi.org/10.48550/arXiv.2605.04251).
- [53] Bui Q C, Scandariato R, Díaz Ferreyra N E. Vul4J: A dataset of reproducible Java vulnerabilities geared towards the study of program repair techniques [C/OL]//Proceedings of the 19th International Conference on Mining Software Repositories. 2022: 464-468. DOI: [10.1145/3524842.3528482](https://doi.org/10.1145/3524842.3528482).
- [54] Wu Y, Jiang N, Pham H V, et al. How effective are neural networks for fixing security vulnerabilities [C/OL]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023: 1282-1294. DOI: [10.1145/3597926.3598135](https://doi.org/10.1145/3597926.3598135).
- [55] Lee H, Zhang Z, Lu H, et al. SEC-bench: Automated benchmarking of LLM agents on real-world software security tasks [C]//Advances in Neural Information Processing Systems: volume 38. 2025.
- [56] Yang J, Zhikhartsev A, Liu Y, et al. Better test cases for better automated program repair [C/OL]//Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering. 2017: 831-841. DOI: [10.1145/3106237.3106274](https://doi.org/10.1145/3106237.3106274).
- [57] Yu Z, Shi W, Sun X, et al. Patch validation in automated vulnerability repair [EB/OL]. 2026. DOI: [10.48550/arXiv.2603.06858](https://doi.org/10.48550/arXiv.2603.06858).
- [58] Shen C, Li J, Mahajan A, et al. PatchBench: Evaluating AI agents for vulnerability patching [EB/OL]. 2026. DOI: [10.48550/arXiv.2609.04075](https://doi.org/10.48550/arXiv.2609.04075).
- [59] Deligiannis P, Lal A, Mehrotra N, et al. RustAssistant: Using LLMs to fix compilation errors in Rust code [C]//2025 IEEE/ACM 47th International Conference on Software Engineering. 2025: 3097-3109.
- [60] Hua B, Ouyang W, Jiang C, et al. Rupair: Towards automatic buffer overflow detection and rectification for Rust [C/OL]//Proceedings of the 37th Annual Computer Security Applications Conference. 2021: 812-823. DOI: [10.1145/3485832.3485841](https://doi.org/10.1145/3485832.3485841).
- [61] Wu Y, Hua B. RUSPATCH: Towards timely and effectively patching Rust applications [C]//2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security. 2023: 517-528.
- [62] Xiang J, He W, Wang X, et al. Evaluating and improving automated repository-level Rust issue resolution with LLM-based agents [C/OL]//Proceedings of the 48th International Conference on Software Engineering. 2026: 805-817. DOI: [10.1145/3744916.3773108](https://doi.org/10.1145/3744916.3773108).

- [63] Jiang R, Wang Y, Dong P, et al. AkiraRust: Re-thinking LLM-aided Rust repair using a feedback-guided thinking switch [EB/OL]. 2026. DOI: [10.48550/arXiv.2602.21681](https://doi.org/10.48550/arXiv.2602.21681).
- [64] The Rust Project. Miri: An interpreter for Rust's mid-level intermediate representation [EB/OL]. 2026. <https://github.com/rust-lang/miri>.
- [65] Tokio Project. Loom: Concurrency permutation testing for Rust [EB/OL]. 2026. <https://docs.rs/loom/latest/loom/>.
- [66] The Rust Project. The Rust reference: Behavior considered undefined [EB/OL]. 2026. <https://doc.rust-lang.org/reference/behavior-considered-undefined.html>.
- [67] Li F, Paxson V. A large-scale empirical study of security patches [C/OL]//Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. 2017: 2201-2215. DOI: [10.1145/3133956.3134072](https://doi.org/10.1145/3133956.3134072).
- [68] Machiry A, Redini N, Camellini E, et al. SPIDER: Enabling fast patch propagation in related software repositories [C/OL]//2020 IEEE Symposium on Security and Privacy. 2020: 1562-1579. DOI: [10.1109/SP40000.2020.00038](https://doi.org/10.1109/SP40000.2020.00038).
- [69] Zhang Z, Zhang H, Qian Z, et al. An investigation of the Android kernel patch ecosystem [C]//30th USENIX Security Symposium. 2021: 3649-3666.
- [70] Red Hat. Security backporting practice [EB/OL]. 2026. <https://access.redhat.com/security/updates/backporting>.
- [71] Zhang H, Qian Z. Precise and accurate patch presence test for binaries [C]//27th USENIX Security Symposium. 2018: 887-902.
- [72] Xu Y, Xu Z, Chen B, et al. Patch based vulnerability matching for binary programs [C/OL]//Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis. 2020: 376-387. DOI: [10.1145/3395363.3397361](https://doi.org/10.1145/3395363.3397361).
- [73] Xu X, Zheng Q, Yan Z, et al. PatchDiscovery: Patch presence test for identifying binary vulnerabilities based on key basic blocks [J/OL]. IEEE Transactions on Software Engineering, 2023, 49(12): 5279-5294. DOI: [10.1109/TSE.2023.3332732](https://doi.org/10.1109/TSE.2023.3332732).
- [74] Jiang Z, Zhang Y, Xu J, et al. PDiff: Semantic-based patch presence testing for downstream kernels [C/OL]//Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security. 2020: 1149-1163. DOI: [10.1145/3372297.3417240](https://doi.org/10.1145/3372297.3417240).
- [75] Yang S, Xu Z, Xiao Y, et al. Towards practical binary code similarity detection: Vulnerability verification via patch semantic analysis [J/OL]. ACM Transactions on Software Engineering and Methodology, 2023, 32(6): 1-29. DOI: [10.1145/3604608](https://doi.org/10.1145/3604608).
- [76] Zhan Q, Hu X, Li Z, et al. PS³: Precise patch presence test based on semantic symbolic signature [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639134](https://doi.org/10.1145/3597503.3639134).

- [77] Zhan Q, Hu X, Xia X, et al. REACT: IR-level patch presence test for binary [C/OL]// Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 381-392. DOI: [10.1145/3691620.3695012](https://doi.org/10.1145/3691620.3695012).
- [78] Dai J, Zhang Y, Jiang Z, et al. BScout: Direct whole patch presence test for Java executables [C]//29th USENIX Security Symposium. 2020: 1147-1164.
- [79] Pan Z, Hu X, Xia X, et al. PPT4J: Patch presence test for Java binaries [C/OL]//Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024: 1-12. DOI: [10.1145/3597503.3639231](https://doi.org/10.1145/3597503.3639231).
- [80] Xie Z, Wen M, Jia H, et al. Precise and efficient patch presence test for Android applications against code obfuscation [C/OL]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023: 347-359. DOI: [10.1145/3597926.3598061](https://doi.org/10.1145/3597926.3598061).
- [81] Xie Z, Wen M, Wei Z, et al. Unveiling the characteristics and impact of security patch evolution [C/OL]//Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering. 2024: 1094-1106. DOI: [10.1145/3691620.3695488](https://doi.org/10.1145/3691620.3695488).
- [82] Hu P, Liang R, Chen K. DeGPT: Optimizing decompiler output with LLM [C/OL]//Network and Distributed System Security Symposium (NDSS). 2024. DOI: [10.14722/ndss.2024.24401](https://doi.org/10.14722/ndss.2024.24401).
- [83] Tan H, Luo Q, Li J, et al. LLM4Decompile: Decompile binary code with large language models [C/OL]//Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024: 3473-3487. DOI: [10.18653/v1/2024.emnlp-main.203](https://doi.org/10.18653/v1/2024.emnlp-main.203).
- [84] Xie D, Zhang Z, Jiang N, et al. ReSym: Harnessing LLMs to recover variable and data structure symbols from stripped binaries [C/OL]//Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. 2024: 4554-4568. DOI: [10.1145/3658644.3670340](https://doi.org/10.1145/3658644.3670340).
- [85] Li S, Zheng Y, Li H, et al. Lares: LLM-driven code slice semantic search for patch presence testing [C/OL]//2025 40th IEEE/ACM International Conference on Automated Software Engineering. 2025: 919-931. DOI: [10.1109/ASE63991.2025.00081](https://doi.org/10.1109/ASE63991.2025.00081).
- [86] Xu C, Luo S, Duan X, et al. PPTSP: Patch presence test via semantic normalization and key path extraction [J/OL]. Software Quality Journal, 2026, 34(2). DOI: [10.1007/s11219-026-09760-9](https://doi.org/10.1007/s11219-026-09760-9).
- [87] Kapoor S, Stroebl B, Siegel Z S, et al. AI agents that matter [EB/OL]. 2024. DOI: [10.48550/arXiv.2407.01502](https://doi.org/10.48550/arXiv.2407.01502).
- [88] Zhang X, Zheng Y, Luo W, et al. A comprehensive empirical analysis of patch presence testing: Capabilities, limitations, and paths forward [J/OL]. Proceedings of the ACM on Software Engineering, 2026, 3(ISSTA): ISSTA053. DOI: [10.1145/3832144](https://doi.org/10.1145/3832144).
- [89] Feng B, Mera A, Lu L. P2IM: Scalable and hardware-independent firmware testing via automatic peripheral interface modeling [C]//29th USENIX Security Symposium. 2020: 1237-1254.

- [90] Wang D, et al. SyzTrust: State-aware fuzzing on trusted OS designed for IoT devices [M]// 2024 IEEE Symposium on Security and Privacy. 2024.
- [91] Zheng Y, Davanian A, Yin H, et al. FIRM-AFL: High-throughput greybox fuzzing of IoT firmware via augmented process emulation [C]//28th USENIX Security Symposium. 2019: 1099-1114.
- [92] Wang X, Li B, Song Y, et al. OpenHands: An open platform for AI software developers as generalist agents [EB/OL]. 2024. DOI: [10.48550/arXiv.2407.16741](https://doi.org/10.48550/arXiv.2407.16741).